

TheAgenticAIEngineer

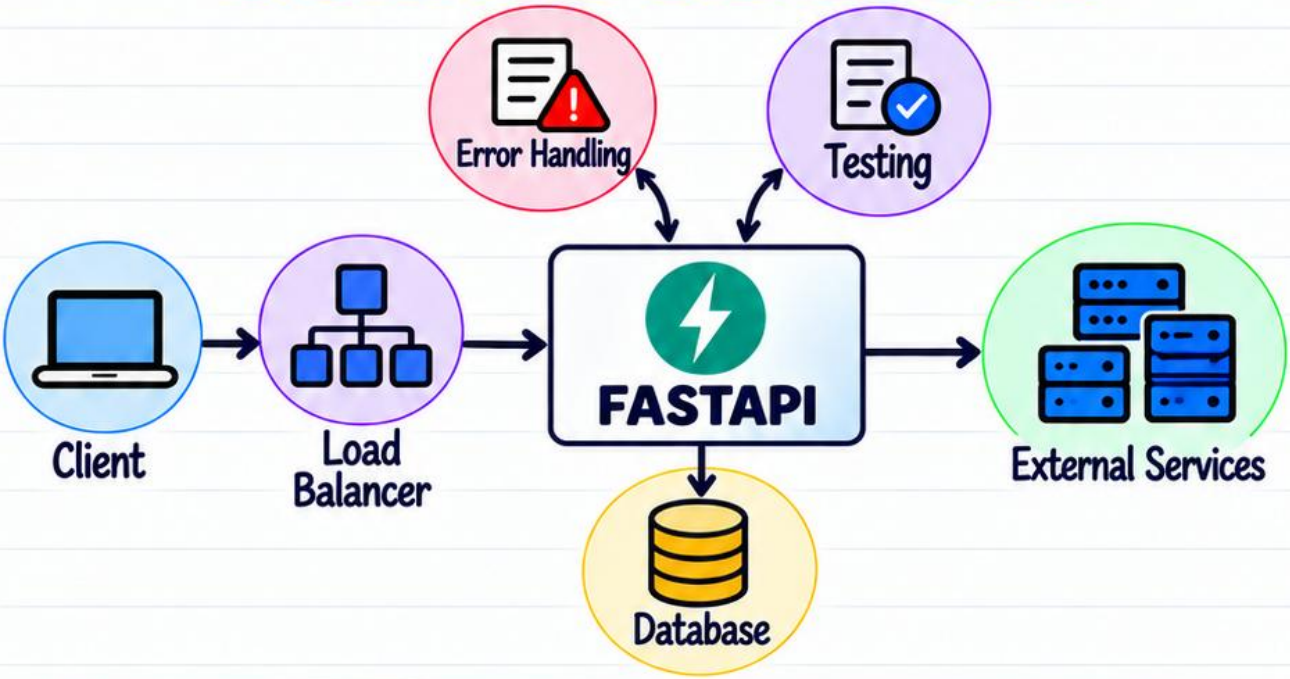
# FASTAPI

## WITH

# ERROR HANDLING & TESTING

## (COMPLETE NOTES)

Complete FastAPI Notes on
Error Handling + Testing Essentials




"From Fundamentals to Production Systems – Step by Step!"

# FastAPI Fundamentals & API Design (Part 1)

## 1 What is FastAPI?

- FastAPI is a modern, high-performance Python web framework for building APIs.
- Created by Sebastián Ramírez.
- Built on top of Starlette (for web) and Pydantic (for data validation).
- Used to create REST APIs quickly and easily.
- Automatically provides interactive documentation (Swagger UI).
- Widely used in AI/ML, RAG and Agentic AI projects to expose LLM/Agent functionalities through APIs.

## 2 Why FastAPI?

- **Fast:** High performance, built on ASGI.
- **Easy to use:** Simple and clean syntax.
- **Automatic docs:** Swagger UI and ReDoc out of the box.
- **Type hints:** Uses Python type hints for better development.
- **Ideal for AI projects:** Easily integrate with LLMs, RAG pipelines and Agents.
- **Production ready:** Used by many companies in real-world applications.

## 3 FastAPI Application Structure

- A FastAPI application is created using the FastAPI() class.
- This is the main application instance.
- All routes and endpoints are defined on this application.
- It acts as the entry point of your API.

main.py

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def root():
    return {"message": "Hello from FastAPI!"}
```

## 4 Routes

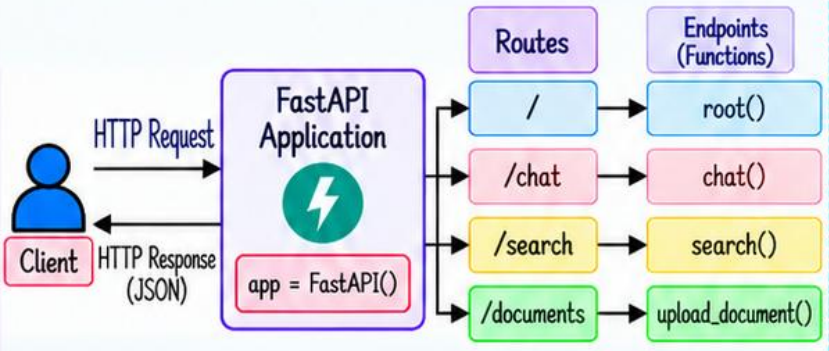
- Routes define the URL path and the HTTP method.
- Routes map a URL to a specific function (endpoint).
- Each route can handle a different task.
- Common HTTP methods: GET, POST, PUT, DELETE, PATCH.
- Routes are added using decorators like `@app.get()`, `@app.post()`, etc.

```
@app.get("/users")  # GET route
def get_users():
    return {"message": "Get all users"}

@app.post("/users")  # POST route
def create_user():
    return {"message": "Create user"}
```

## 5 Endpoints

- An endpoint is the actual function that handles the request and returns a response.
- Each endpoint is associated with a route.
- An endpoint can take input (path, query, body) and return output (JSON, text, etc).
- Endpoints contain the business logic.
- In RAG/Agentic AI projects, endpoints are used for chat, document upload, search, etc.



# FastAPI Fundamentals & API Design (Part 2)

## 6 HTTP Methods

- HTTP methods define what action to perform on a resource (e.g., get, create, update, delete).
- Used in API routes to handle different tasks.
- FastAPI supports all standard HTTP methods.
- Each method has a specific use case.

| Method | Purpose | Example | Use Case |
|---|---|---|---|
| GET | Read data | /users | Get all users |
| POST | Create data | /users | Create a new user |
| PUT | Update (Full) | /users/1 | Update all fields of a user |
| PATCH | Update (Partial) | /users/1 | Update specific fields |
| DELETE | Delete data | /users/1 | Delete a user |

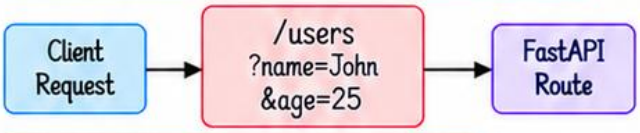
**Example: Different HTTP Methods in FastAPI**

```
@app.get("/users")
def get_users():
    return {"message": "Get all users"}
@app.post("/users")
def create_user():
    return {"message": "Create a user"}
@app.put("/users/1")
def update_user():
    return {"message": "Update user"}
@app.delete("/users/1")
def delete_user():
    return {"message": "Delete user"}
```

## 8 Query Parameters

- Query parameters are optional values in the URL.
- Used for filtering, searching, pagination, etc.
- Can have default values.
- FastAPI automatically handles type conversion and validation.

**URL Format:** /users?name=John&age=25




**Example: Query Parameters in FastAPI**

```
@app.get("/users")
def get_users(name: str = None, age: int = None):
    return {"name": name, "age": age}
```

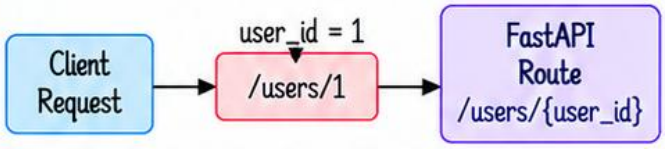
**Key Points:**

- Added to URL after ? as key=value.
- Can be optional with default values.
- Useful for filters like search, pagination, sorting.

## 7 Path Parameters

- Path parameters are part of the URL path.
- Used to pass dynamic values to the endpoint.
- FastAPI automatically validates the type.
- If the resource is not found, return 404 error.

**URL Format:** /users/{user_id}




**Example: Path Parameter in FastAPI**

```
@app.get("/users/{user_id}")
def get_user(user_id: int):  # user_id is int type
    return {"user_id": user_id, "name": "John"}
```

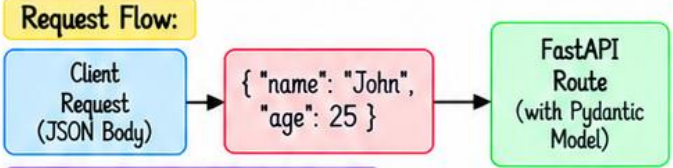
**Key Points:**

- Defined using {parameter_name} in the path.
- Specify the type (int, str, etc.) for validation.
- If the user_id does not exist, use HTTPException (404).
- Common types: int, str, float, slug, uuid.

## 9 Request Body

- Request body is used to send data in the request.
- Commonly used with POST, PUT, PATCH.
- We define a Pydantic model (BaseModel) for validation.
- FastAPI automatically validates the input data.
- If data is invalid, it returns a 422 Unprocessable Entity error.

**Request Flow:**




**Example: Request Body in FastAPI**

```
from pydantic import BaseModel
class User(BaseModel):
    name: str
    age: int
@app.post("/users")
def create_user(user: User):  # request body as User model
    return {"name": user.name, "age": user.age}
```

**Key Points:**

- Use Pydantic BaseModel to define request body schema.
- FastAPI validates the data automatically.
- Used mainly with POST, PUT, PATCH methods.
- Returns 422 error if the input does not match the model.

# FastAPI Fundamentals & API Design (Part 3)

## 11 HTTP Status Codes

- HTTP status codes tell the result of an API request.
- FastAPI returns appropriate status codes automatically.
- We can also set custom status codes in our endpoints.
- Common categories: 1xx (Informational), 2xx (Success), 4xx (Client Error), 5xx (Server Error).

| Code | Meaning | Example Use Case |
|---|---|---|
| 200 OK | Request successful | Get data successfully |
| 201 Created | Resource created | Create user, upload file |
| 204 No Content | Successful but no data | Delete user |
| 400 Bad Request | Invalid request | Missing or wrong input |
| 401 Unauthorized | Authentication required | No token / invalid token |
| 403 Forbidden | Access denied | Does not have permission |
| 404 Not Found | Resource not found | User/ID not found |
| 422 Unprocessable Entity | Validation error | Invalid request body |
| 500 Internal Server Error | Server error | Unexpected error in API |

## 12 HTTPException

- HTTPException is used to raise errors in FastAPI.
- It stops the request and returns an error response with a status code.
- We can add a custom message and optional details.
- Commonly used for 400, 401, 403, 404, etc.

**Syntax:**

```python
from fastapi import HTTPException
raise HTTPException(
    status_code=404,
    detail="User not found"
)
```

**Example: Get user with error handling**

```python
from fastapi import HTTPException
@app.get("/users/{user_id}")
def get_user(user_id: int):
    user = get_user_from_db(user_id)
    if not user:
        raise HTTPException(
            status_code=404,
            detail="User not found"
        )
    return {"id": user.id, "name": user.name}
```

**Error Response (JSON)**

```
{
  "detail": "User not found"
}
```

Status Code: 404

## 13 Swagger / ReDoc

- FastAPI automatically generates interactive API documentation.
- Swagger UI is available at /docs.
- ReDoc is available at /redoc.
- These docs show all routes, request bodies, parameters and allow testing the APIs.
- Very useful for development, testing and sharing APIs.

**Swagger UI (/docs)**




- Interactive UI
- Test APIs directly from browser
- Shows request/response schema

**ReDoc (/redoc)**




- Clean and simple UI
- Good for large APIs
- Better for reading documentation

## 14 APIRouter

- APIRouter helps to organize routes in a modular way.
- We can split routes into different files (e.g., user routes, chat routes).
- Makes the code clean and easy to maintain.
- We can set a common prefix, tags, and dependencies for all routes in a router.

**Example Project Structure**

```
app/
├── main.py
├── routers/
│   ├── user.py
│   └── chat.py
└── schemas/
    └── user.py
```

**user.py (Router file)**

```python
from fastapi import APIRouter
router = APIRouter(
    prefix="/users",
    tags=["Users"]
)
@router.get("/")
def get_users():
    return {"message": "Get all users"}
```

**chat.py (Router file)**

```python
from fastapi import APIRouter
router = APIRouter(
    prefix="/chat",
    tags=["Chat"]
)
@router.post("/")
def chat():
    return {"message": "Chat with AI"}
```

**main.py (Include routers)**

```python
from fastapi import FastAPI
from app.routers import user, chat

app = FastAPI()

app.include_router(user.router)
app.include_router(chat.router)
```

# Pydantic & Request/Response Validation (Part 1)

## 1 What is Pydantic BaseModel?

- BaseModel is a class from Pydantic used to define the structure of data.
- It automatically validates the incoming data.
- It converts data to Python types (type conversion).
- It gives clear error messages if data is invalid.
- It is used for both request and response models in FastAPI.

**Example: Basic BaseModel**

```python
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int
    email: str
```

**Key Points:**

- Define field name and type.
- Automatically validates data.
- Converts data types (e.g. "25" → 25).

## 2 Request Schemas

- Request schema defines the expected structure of data sent by the client.
- Used with POST, PUT, PATCH endpoints.
- Validates incoming JSON data automatically.
- FastAPI returns 422 error if data is invalid.
- We use BaseModel to create request schema.

**Example: Request Schema**

```python
from pydantic import BaseModel, EmailStr
class CreateUserRequest(BaseModel):
    name: str
    age: int
    email: EmailStr
```




## 3 Response Schemas

- Response schema defines the structure of data returned to the client.
- Used with response_model in FastAPI.
- Ensures only defined fields are returned.
- Automatically converts Python objects to JSON.
- Helps maintain a consistent API response format.

**Example: Response Schema**

```python
from pydantic import BaseModel
class UserResponse(BaseModel):
    id: int
    name: str
    age: int
    email: str
```

**Key Points:**

- Use response_model in route.
- Only defined fields are returned.
- Useful for hiding internal fields (e.g. password).

**Example: Using Request and Response Schema in FastAPI**

```python
from fastapi import FastAPI
from pydantic import BaseModel, EmailStr

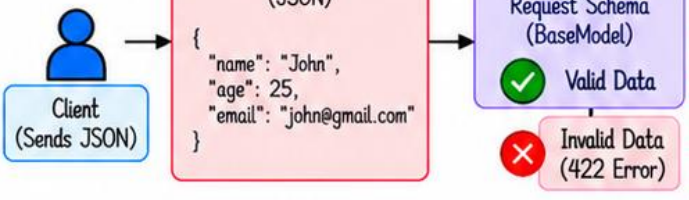
app = FastAPI()

class CreateUserRequest(BaseModel):
    name: str
    age: int
    email: EmailStr

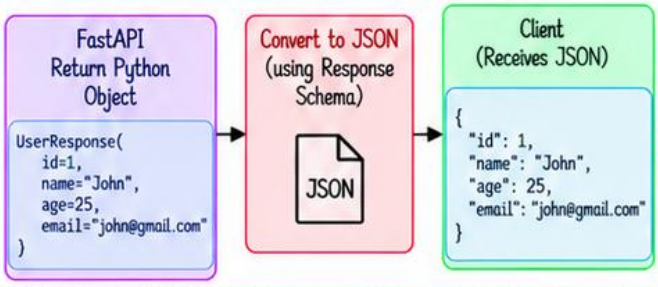
class UserResponse(BaseModel):
    id: int
    name: str
    age: int
    email: str

@app.post("/users", response_model=UserResponse)
def create_user(user: CreateUserRequest):
    # In real app, you would save to database
    return UserResponse(id=1, name=user.name,
                        age=user.age, email=user.email)
```

**Request Flow (Validation)**




**Response Flow (Serialization)**



# Pydantic & Request/Response Validation (Part 2)

## 4 response_model

- response_model tells FastAPI what data to return in the response.
- Only the fields defined in the response model are sent to the client.
- Helps to hide sensitive fields (e.g. password).
- Automatically converts Python objects to JSON.
- Improves API documentation (Swagger UI).

**Example: Using response_model**

```
from fastapi import FastAPI
from pydantic import BaseModel

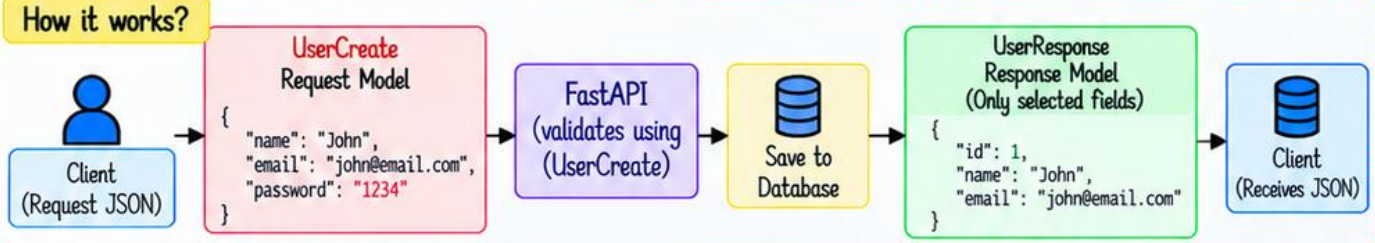
app = FastAPI()

Request Model (UserCreate)
class UserCreate(BaseModel):
    name: str
    email: str
    password: str

Response Model (UserResponse)
class UserResponse(BaseModel):
    id: int
    name: str
    email: str

@app.post("/users", response_model=UserResponse)
def create_user(user: UserCreate):
    # In real app, save to DB and return user data
    return UserResponse(id=1, name=user.name, email=user.email)
```

**How it works?**




## 5 Field Validation

- Pydantic automatically validates data types (str, int, email, etc.).
- We can add extra validation rules using Field() or validators.
- It ensures the input data is correct before it reaches our logic.
- If validation fails, FastAPI returns 422 Unprocessable Entity error.

**Example: Field Validation**

```
from pydantic import BaseModel, Field, EmailStr

class UserCreate(BaseModel):
    name: str = Field(..., min_length=3, max_length=50,
                      description="User name")
    age: int = Field(..., ge=18, le=100,
                     description="User age (18-100)")
    email: EmailStr = Field(..., description="Valid email")
```

**Validation Rules Example:**

| Parameter | Meaning | Example |
|---|---|---|
| min_length | Minimum length of string | min_length=3 |
| max_length | Maximum length of string | max_length=50 |
| ge | Greater than or equal (for numbers) | ge=18 |
| le | Less than or equal (for numbers) | le=100 |
| EmailStr | Validates email format | email: EmailStr |
| ... | Other options like regex, gt, lt, etc. | regex="^[A-Za-z]+$" |

**Error Response (Invalid Data)**

If age is less than 18:

```
{
  "detail": [
    {
      "loc": ["body", "age"],
      "msg": "ensure this value is greater than or equal to 18",
      "type": "value_error.number.not_ge",
      "ctx": {"limit_value": 18}
    }
  ]
}
```

FastAPI automatically returns 422 error with details about what went wrong.

## 6 Optional / Default Fields

- Fields can be made optional using Optional[...].
- We can provide default values for fields.
- Useful when some fields are not mandatory in the request.
- Pydantic automatically handles missing fields using default values.

**Example: Optional and Default Fields**

```
from pydantic import BaseModel, Field
from typing import Optional, List
class UserCreate(BaseModel):
    name: str
    age: Optional[int] = None                # optional field
    role: str = "user"                       # default value
    is_active: bool = True                   # default value
    tags: Optional[List[str]] = []           # optional list with default
```

**Example Request JSON (Some fields are optional)**

```
{
  "name": "John",
  "age": 25,
  "tags": ["AI", "RAG"]
  // role and is_active will use default values
}
```

**Parsed Python Object**

```
{
  "name": "John",
  "age": 25,
  "role": "user",
  "is_active": true,
  "tags": ["AI", "RAG"]
}
```

# Pydantic & Request/Response Validation (Part 3)

## 7 Nested Models

- Nested models allow one Pydantic model to contain another model.
- Useful for complex data structures (e.g., user with address, order with items).
- FastAPI automatically validates all nested fields.
- Used in both request and response schemas.

**Real World Example**

A user has an `Address` model inside a `User` model.

**Example: Nested Models in FastAPI**

```python
from fastapi import FastAPI
from pydantic import BaseModel

class Address(BaseModel):
    street: str
    city: str
    pincode: str

class UserCreate(BaseModel):
    name: str
    age: int
    address: Address  # nested model
```

**Request JSON (Nested)**

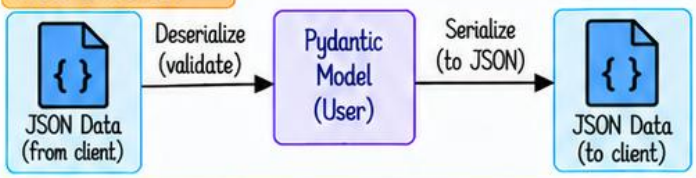
```json
{
  "name": "John",
  "age": 25,
  "address": {
    "street": "MG Road",
    "city": "Bangalore",
    "pincode": "560001"
  }
}
```

Client (Sends JSON) → FastAPI Validates Nested Model (UserCreate) → Parsed Python Object (dict)

## 8 Serialization / Deserialization

- Serialization: Convert Python object (Pydantic model) to JSON (dict) for API response.
- Deserialization: Convert incoming JSON data to Pydantic model for validation.
- FastAPI automatically handles both using Pydantic.
- We can also manually convert using `.model_dump()` and `.model_validate()`.

**How it works?**




**Example: Serialization and Deserialization**

```python
from pydantic import BaseModel

class User(BaseModel):
    id: int
    name: str
    age: int
```

**model_validate()**
Converts JSON/dict to Pydantic model (with validation).

**1. Deserialization (JSON → Pydantic model)**

```python
user_data = {"id": 1, "name": "John", "age": 25}
user = User.model_validate(user_data)
# or: user = User(**user_data)
```

**2. Serialization (Pydantic model → JSON)**

```python
user = User(id=1, name="John", age=25)
user_dict = user.model_dump()  # dict
user_json = user.model_dump_json()  # JSON string
```

**model_dump()**
Converts Pydantic model to dict/JSON (for response).

## 9 Validation Errors

- If the input data does not match the schema, FastAPI returns 422 Unprocessable Entity error.
- It shows clear error messages about which field is invalid.
- Validation happens automatically using Pydantic.
- Helps to catch issues before the data reaches our business logic.
- Useful for maintaining data quality and security.

**Common Reasons for Validation Errors**

- Missing required fields.
- Wrong data types (e.g., string instead of int).
- Invalid value (e.g., age < 0, email format wrong).
- Nested model fields mismatch.

**Example: Validation Error in FastAPI**

```python
from pydantic import BaseModel, Field

class UserCreate(BaseModel):
    name: str = Field(..., min_length=3)
    age: int = Field(..., ge=18)
    email: str = Field(..., pattern=r"^[^@]+@[^@]+\.[^@]+$")
```

**Invalid Request JSON (age is 15)**

```json
{
  "name": "Jo",
  "age": 15,
  "email": "john"
}
```

FastAPI Validation (Fails) →

**Error Response (422)**

```json
{
  "detail": [
    {
      "loc": ["body", "age"],
      "msg": "Input should be greater than or equal to 18",
      "type": "greater_than_equal"
    }
  ]
}
```

**Key Points:**

- FastAPI returns 422 for validation errors.
- Error response contains field name (loc) and clear message (msg).
- We can add custom validation rules using Field(...).

# Dependency Injection & Authentication (Part 1)

## 1 What is Depends()?

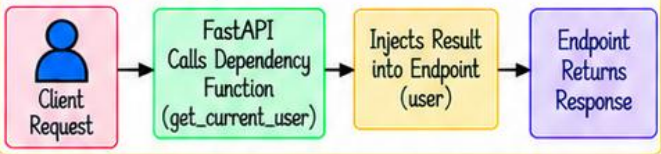
- Depends() is a special feature in FastAPI used for Dependency Injection.
- It allows us to reuse common logic across multiple endpoints.
- FastAPI automatically calls the dependency function before the endpoint function.
- Helps in keeping the code clean and modular.
- Used for database connection, authentication, permission checks, etc.

### Basic Example

```python
from fastapi import FastAPI, Depends
app = FastAPI()
def get_current_user():
    return {"name": "John"}

@app.get("/profile")
def get_profile(user = Depends(get_current_user)):
    return {"message": f"Hello {user['name']}"}
```

### How it works?




## 2 Reusable Dependencies

- We can create reusable dependency functions.
- The same dependency can be used in multiple endpoints.
- Helps reduce code duplication.
- Useful for common logic like authentication, logging, header extraction, etc.

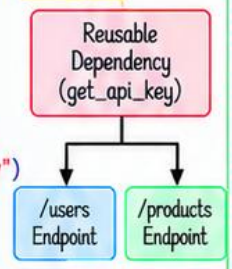
### Example: Reusable Dependency

```python
from fastapi import FastAPI, Depends
app = FastAPI()
def get_api_key(x_api_key: str):
    if x_api_key != "mysecretkey":
        raise HTTPException(status_code=401,
                            detail="Invalid API Key")
    return x_api_key

@app.get("/users")
def get_users(api_key: str = Depends(get_api_key)):
    return {"message": "Get all users", "api_key": api_key}

@app.get("/products")
def get_products(api_key: str = Depends(get_api_key)):
    return {"message": "Get all products", "api_key": api_key}
```

### Usage




## 3 Database Dependency

- Used to manage database connections.
- Automatically creates a database session for each request.
- Yields the session to the endpoint function.
- Closes the session after the request is completed.
- Helps maintain clean and safe database operations.

### Example: Database Dependency (SQLAlchemy)

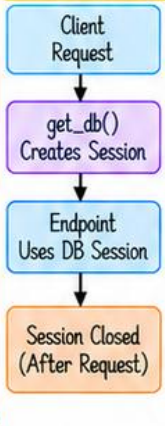
```python
from fastapi import Depends, FastAPI
from sqlalchemy.orm import Session
from database import SessionLocal # SQLAlchemy session

app = FastAPI()

def get_db():
    db = SessionLocal()   # create session
    try:
        yield db       # provide session to endpoint
    finally:
        db.close()     # close session after request

@app.get("/users")
def get_users(db: Session = Depends(get_db)):
    users = db.query(User).all()
    return {"users": [u.name for u in users]}
```

### How it works?




## 4 Authentication Dependency

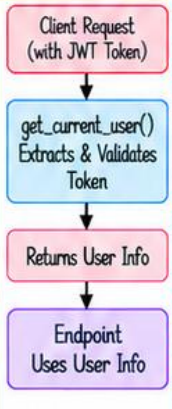
- Used to get the current logged-in user from a token (e.g., JWT).
- Extracts and validates the token from the request.
- Returns the user information if the token is valid.
- Can be reused in multiple protected endpoints.

### Example: Authentication Dependency (JWT)

```python
from fastapi import Depends, FastAPI, HTTPException
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
from jose import JWTError, jwt
app = FastAPI()
security = HTTPBearer()
SECRET_KEY = "your_secret_key"

async def get_current_user(credentials: HTTPAuthorizationCredentials
                           = Depends(security)):
    token = credentials.credentials
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        user_id = payload.get("sub")
        if not user_id:
            raise HTTPException(status_code=401, detail="Invalid token")
        return {"id": user_id, "name": payload.get("name")}
    except JWTError:
        raise HTTPException(status_code=401, detail="Token is invalid")
@app.get("/me")
async def read_me(user = Depends(get_current_user)):
    return {"message": f"Hello {user['name']}", "id": user["id"]}
```

### How it works?



# Dependency Injection & Authentication (Part 2)

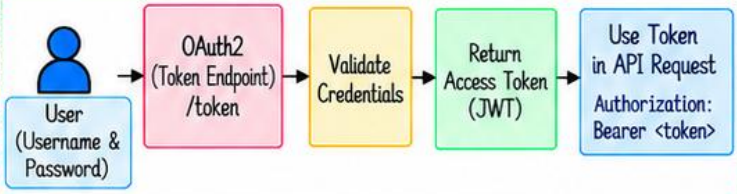
## 5 What is OAuth2 ?

- OAuth2 is an open standard for authentication and authorization.
- It allows a user to give limited access to their resources without sharing their password.
- It uses tokens to access protected APIs.
- Widely used in web and mobile applications.
- Commonly used with JWT in FastAPI.

### Why OAuth2 ?

- Secure way to authenticate users.
- No need to share password with the API.
- Supports different grant types (e.g., password, authorization code).
- Works well with JWT tokens.
- Standard and widely adopted.

### How OAuth2 Works? (Password Flow Example)




### Example: OAuth2 Password Flow in FastAPI

```python
from fastapi import FastAPI, Depends
from fastapi.security import OAuth2PasswordBearer

app = FastAPI()

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/token")

@app.get("/protected")
def protected_route(token: str = Depends(oauth2_scheme)):
    return {"message": "You are authenticated",
            "token": token}
```

**Key Points:**

- OAuth2PasswordBearer tells FastAPI to expect a token from /token.
- Token is read from Authorization header.
- Integrates with JWT for secure authentication.

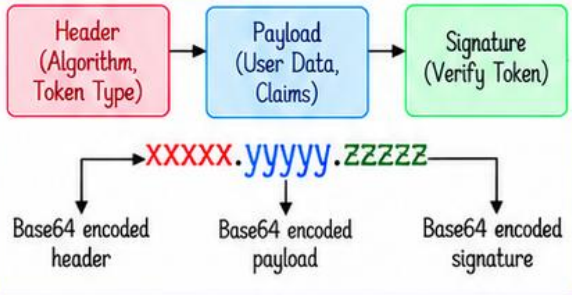
## 6 What is JWT ?

- JWT (JSON Web Token) is a compact, URL-safe token used to securely transmit information between parties.
- It contains user information in a signed token.
- Used with OAuth2 for authentication in FastAPI.
- Token is verified on every request.
- No need to store session data on the server.

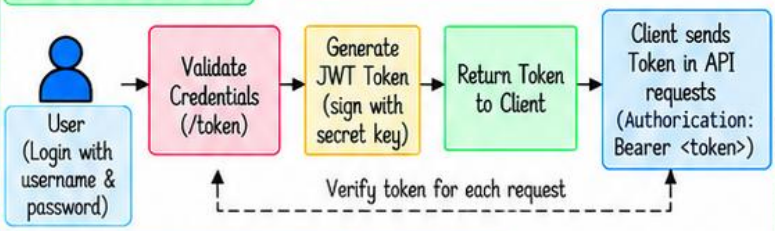
### Why JWT ?

- Stateless authentication (no server-side session).
- Secure (signed using a secret key).
- Easy to verify and use in APIs.
- Contains user information (e.g., user id, role).
- Widely used in modern web applications.

### Structure of JWT




### How JWT Works?




### Example: Create and Verify JWT in FastAPI

#### 1. Create JWT Token (after login)

```python
from jose import JWTError, jwt
from datetime import datetime, timedelta
SECRET_KEY = "your_secret_key"
ALGORITHM = "HS256"
ACCESS_TOKEN_EXPIRE_MINUTES = 30

def create_access_token(data: dict):
    to_encode = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    to_encode.update({"exp": expire})
    token = jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
    return token
```

**Key Points:**

- JWT is signed using a secret key.
- Contains user data (e.g., user id).
- Has an expiration time (exp).
- Uses algorithm like HS256.

#### 2. Verify JWT Token (using dependency)

```python
from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/token")
def get_current_user(token: str = Depends(oauth2_scheme)):
  try:
      payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
      user_id: str = payload.get("sub")
      if user_id is None:
          raise HTTPException(status_code=401, detail="Invalid token")
      return {"id": user_id}
  except JWTError:
      raise HTTPException(status_code=401, detail="Token is invalid")
```

**Key Points:**

- Reads token from Authorization header.
- Decodes and verifies token.
- Extracts user information.
- Raises 401 if token is invalid or expired.

# Dependency Injection & Authentication (Part 3)

## 7 Protected Endpoints

- Protected endpoints can only be accessed by authenticated users.
- We use Depends() with authentication dependency (e.g., OAuth2 or API key).
- If the user is not authenticated, FastAPI returns 401 Unauthorized.
- We can protect single or multiple endpoints.
- Used for secure access to sensitive data.

**Example: Protected Endpoint using JWT (OAuth2)**

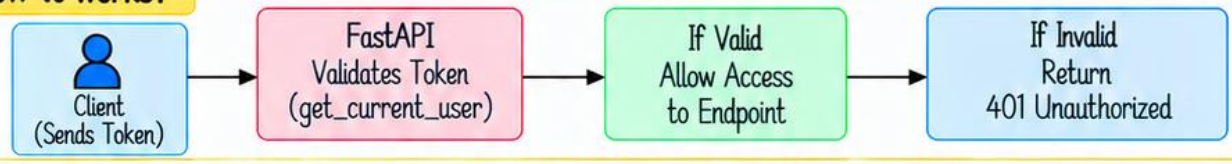
```python
from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import OAuth2PasswordBearer

app = FastAPI()
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/token")

def get_current_user(token: str = Depends(oauth2_scheme)):
    # In real app, decode and verify token
    if not token or token != "valid_token":
        raise HTTPException(status_code=401, detail="Not authenticated")
    return {"username": "john", "role": "user"}

@app.get("/protected")
def protected_route(user: dict = Depends(get_current_user)):
    return {"message": f"Hello {user['username']}, this is a protected endpoint"}
```

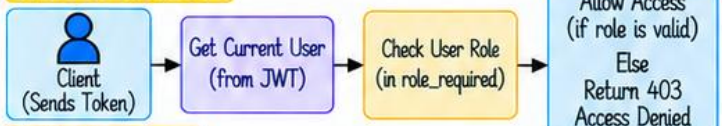
**How it works?**




## 8 RBAC (Role-Based Access Control)

- RBAC controls what a user can do based on their role (e.g., admin, manager, user).
- Each role has specific permissions.
- We check the user role inside a dependency.
- Helps secure APIs and restrict access to sensitive endpoints.
- Common roles: admin (full access), manager (limited access), user (basic access).

**How it works?**




**Role Example**

| Role | Permissions (Example) |
|---|---|
| admin | Access all endpoints (create, read, update, delete) |
| manager | Access limited endpoints (read, update) |
| user | Access only basic endpoints (read) |

**Example: RBAC using Dependency**

```python
from fastapi import FastAPI, Depends, HTTPException
from typing import List

app = FastAPI()

def role_required(allowed_roles: List[str]):
    def check_role(user: dict = Depends(get_current_user)):
        if user.get("role") not in allowed_roles:
            raise HTTPException(status_code=403, detail="Access denied")
        return user
    return check_role

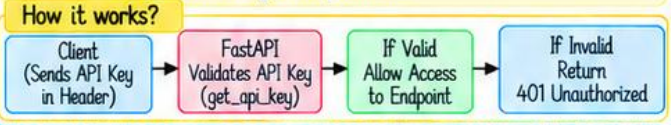
@app.get("/admin-only")
def admin_route(user: dict = Depends(role_required(["admin"]))):
    return {"message": "Welcome Admin"}

@app.get("/manager-or-admin")
def manager_route(user: dict = Depends(role_required(["admin","manager"]))):
    return {"message": "Welcome Manager/Admin"}
```

## 9 API Key Authentication

- API key authentication uses a secret key passed in the request header or query parameter.
- Used for server-to-server communication or internal APIs.
- FastAPI provides APIKeyHeader and APIKeyQuery.
- If the key is invalid or missing, returns 401 error.
- Simple and easy to implement.

**How it works?**




**Example: API Key Authentication (Header)**

```python
from fastapi import FastAPI, Depends, HTTPException
from fastapi.security import APIKeyHeader
app = FastAPI()
api_key_header = APIKeyHeader(name="X-API-Key")

def get_api_key(api_key: str = Depends(api_key_header)):
    if api_key != "mysecretkey":
        raise HTTPException(status_code=401, detail="Invalid API Key")
    return api_key
@app.get("/secure-data")
def get_data(api_key: str = Depends(get_api_key)):
    return {"message": "This is secure data"}
```

**Request Example**

```
GET /secure-data
Headers:
X-API-Key: mysecretkey
```

**Response Example**

```
{
  "message": "This is secure data"
}
```

# Async & Streaming (Part 1)

## 1 async / await

- **async** and **await** are keywords in Python used to write asynchronous code.
- **async** is used to define an asynchronous function (coroutine).
- **await** is used to wait for an async operation to complete without blocking the program.
- Helps handle multiple tasks at the same time (like multiple API calls, DB calls).
- Useful for I/O-bound operations (network requests, database queries, file operations).

### Example: async / await in FastAPI

```python
from fastapi import FastAPI
import asyncio

app = FastAPI()

@app.get("/async")
async def fetch_data():
    await asyncio.sleep(2)   # simulate slow task
    return {"message": "Data fetched after 2 seconds"}
```

**async def →** defines an async function

**await →** waits for the task to complete

## 2 Sync vs Async

- Synchronous (Sync) code runs one task at a time. It waits for a task to complete before starting the next one.
- Asynchronous (Async) code can handle multiple tasks at the same time without waiting.
- Async is more efficient for I/O-bound tasks (like API calls, database calls).
- Sync is simpler but can be slow when tasks take time.
- FastAPI supports both sync and async endpoints.

### Example: Sync vs Async in FastAPI

**Synchronous (Sync)**

```python
from fastapi import FastAPI
import time

app = FastAPI()

@app.get("/sync")
def sync_endpoint():
    time.sleep(2)  # blocks
    return {"message": "Done"}
```

**Asynchronous (Async)**

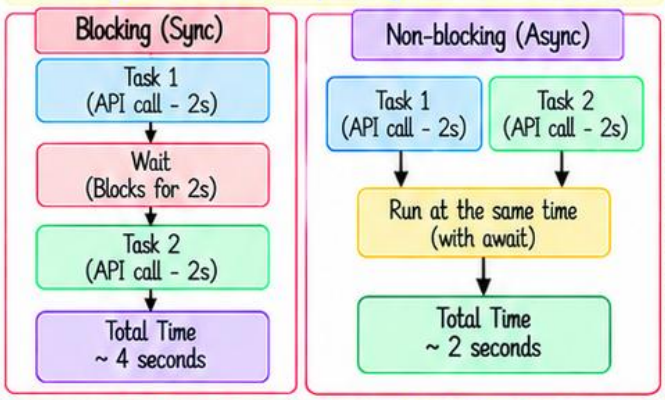
```python
from fastapi import FastAPI
import asyncio

app = FastAPI()

@app.get("/async")
async def async_endpoint():
    await asyncio.sleep(2)
    return {"message": "Done"}
```

## 3 Blocking vs Non-blocking Operations

- Blocking operation stops the program and waits until the task is complete.
- Non-blocking operation allows the program to continue and handle other tasks while waiting.
- Sync functions usually perform blocking operations.
- Async functions perform non-blocking operations using await.
- Common blocking operations: time.sleep(), file I/O, traditional HTTP requests (requests).
- Async uses non-blocking operations like asyncio.sleep(), httpx, async database calls.




## 4 Real Example: Multiple API Calls

- In real applications, we often need to call multiple APIs (e.g., user service, product service).
- Using async and await, we can call multiple APIs concurrently to reduce total time.
- This improves performance and user experience.

### Example: Fetch multiple APIs concurrently

```python
from fastapi import FastAPI
import httpx
import asyncio

app = FastAPI()

async def get_user():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.example.com/user")
        return response.json()

async def get_products():
    async with httpx.AsyncClient() as client:
        response = await client.get("https://api.example.com/products")
        return response.json()

@app.get("/data")
async def get_data():
    # Run both API calls at the same time
    user_task = get_user()
    product_task = get_products()
    user_data, product_data = await asyncio.gather(user_task, product_task)
    return {"user": user_data, "products": product_data}
```

**asyncio.gather()** runs multiple async tasks concurrently.

# Async & Streaming (Part 2)

## 5 Async API Calls

- Used to call external APIs asynchronously without blocking the application.
- Allows multiple API calls to run concurrently (using asyncio / httpx).
- Improves performance for I/O-bound tasks.
- Useful when calling multiple services (e.g., user service, product service, etc.).
- FastAPI endpoints can be async and await these calls.

### How it works?

FastAPI Endpoint (async) → Async HTTP Client (httpx) → Call Multiple External APIs Concurrently → Combine Results (Return JSON)

### Example: Async API Calls using httpx

```python
from fastapi import FastAPI
import httpx, asyncio

app = FastAPI()

async def fetch_user(user_id: int):
    async with httpx.AsyncClient() as client:
        response = await client.get(f"https://api.example.com/users/{user_id}")
        return response.json()

async def fetch_orders(user_id: int):
    async with httpx.AsyncClient() as client:
        response = await client.get(f"https://api.example.com/orders?user_id={user_id}")
        return response.json()

@app.get("/user-info/{user_id}")
async def get_user_info(user_id: int):
    user_task = fetch_user(user_id)
    order_task = fetch_orders(user_id)
    user_data, order_data = await asyncio.gather(user_task, order_task)
    return {"user": user_data, "orders": order_data}
```

### Key Points:

- Use httpx.AsyncClient() for async HTTP calls.
- Run multiple calls concurrently using asyncio.gather().
- Useful for microservices or third-party APIs.
- Handle errors and timeouts in production.

## 6 Async Database Operations

- Used to perform database operations asynchronously.
- Helps handle multiple database queries concurrently.
- Suitable for I/O-bound operations (like DB access).
- In FastAPI, we can use async database clients like SQLAlchemy (async) or databases package.
- Improves performance and scalability for high concurrent users.

### How it works?

FastAPI Endpoint (async) → Async DB Session (AsyncEngine) → Execute Async Queries (await) → Return Response (JSON)

### Example: Async Database Operations (SQLAlchemy)

```python
from fastapi import FastAPI, Depends
from sqlalchemy.ext.asyncio import AsyncSession, create_async_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy import select
from models import User  # SQLAlchemy model

DATABASE_URL ="postgresql+asyncpg://user:password@localhost:5432/mydb"
engine = create_async_engine(DATABASE_URL, echo=True)
AsyncSessionLocal = sessionmaker(engine, class_=AsyncSession, expire_on_commit=False)

# Dependency to get async DB session
async def get_db():
    async with AsyncSessionLocal() as session:
        yield session

app = FastAPI()

@app.get("/users")
async def get_users(db: AsyncSession = Depends(get_db)):
    result = await db.execute(select(User))
    users = result.scalars().all()
    return [{"id": u.id, "name": u.name, "email": u.email} for u in users]
```

### Key Points:

- Use async engine (create_async_engine).
- Use AsyncSession with Depends().
- Use await for DB operations.
- Works with PostgreSQL (asyncpg), MySQL (aiomysql), etc.

## 7 StreamingResponse

- Used to stream data (large files, real-time data, LLM responses) instead of sending a full response at once.
- Sends data in chunks to the client.
- Improves user experience for long-running tasks.
- Useful for file downloads, real-time logs, or streaming LLM tokens.
- FastAPI provides StreamingResponse for this purpose.

### How it works?

Generate Data in Chunks (iterable/async generator) → StreamingResponse (streams chunks) → Client Receives Data Incrementally (in real time)

### Example 1: Stream a Large File (Download)

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()
def file_iterator(file_path: str, chunk_size: int = 1024 * 1024):
    with open(file_path, "rb") as file:
        while chunk := file.read(chunk_size):
            yield chunk

@app.get("/download")
def downlaad_file():
    return StreamingResponse(
        file_iterator("data/large_file.zip"),
        media_type="application/zip",
        headers={"Content-Disposition": "attachment; filename=large_file.zip"}
    )
```

### Example 2: Stream LLM-like Tokens (Real-time Response)

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import asyncio

app = FastAPI()
async def token_generator():
    for word in ["Hello", " ", "this", " ", "is", " ", "a", " ", "streaming", " ", "response", "!"]:
        yield chunk
        await asyncio.sleep(0.2)  # simulate token delay

@app.get("/streamd")
async def stream_response():
    return StreamingResponse(token_generator(), media_type="text/plain")
```

### Key Points:

- Sends data in small chunks.
- Memory efficient.
- Useful for large files and real-time responses.
- Supports streaming LLM responses, logs, etc.

# Async & Streaming (Part 3)

## 8 Server-Sent Events (SSE)

- SSE (Server-Sent Events) allows the server to push real-time updates to the client over a single HTTP connection.
- It is a one-way streaming communication (from server → client).
- Simple and lightweight (uses text/event-stream).
- Automatically reconnects if the connection is lost.
- Useful for real-time updates, progress tracking, logs, notifications, and LLM/agent streaming.

### How it works?




### Example: SSE in FastAPI

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import asyncio

app = FastAPI()

async def event_generator():
    for i in range(1, 6):
        data = f"data: Progress {i}/5\n\n"
        yield data
        await asyncio.sleep(1)  # simulate work

@app.get("/events")
async def sse_events():
    return StreamingResponse(
        event_generator(),
        media_type="text/event-stream"
    )
```

### Key Points:

- Uses text/event-stream media type.
- Each message starts with **data:** and ends with **\n\n**.
- One-way streaming (server → client).
- Auto-reconnect support in browsers.
- Useful for real-time UIs, progress updates, logs.

### Client (Browser) Example

```javascript
const eventSource = new EventSource("/events");

eventSource.onmessage = (event) => {
  console.log("Update:", event.data);
};
```

## 9 LLM Token Streaming

- LLM token streaming sends the generated response token-by-token instead of waiting for the full response.
- Improves user experience (faster time to first token).
- Uses StreamingResponse (SSE or plain text).
- Commonly used in chat applications and AI assistants.
- Works with local LLMs (Ollama, vLLM) or APIs (OpenAI, etc.).

### How it works?




### Example: LLM Token Streaming (OpenAI / Local LLM)

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import openai

app = FastAPI()

async def stream_llm_response(prompt: str):
    client = openai.AsyncOpenAI(api_key="YOUR_API_KEY")
    response = await client.chat.completions.create(
        model="gpt-4o-mini",  # or your local model
        messages=[{"role": "user", "content": prompt}],
        stream=True
    )
    async for chunk in response:
        if chunk.choices[0].delta.content:
            token = chunk.choices[0].delta.content
            yield f"data: {token}\n\n"  # SSE format

@app.get("/chat/stream")
async def chat_stream(prompt: str):
    return StreamingResponse(
        stream_llm_response(prompt),
        media_type="text/event-stream"
    )
```

### Key Points:

- Uses **stream=True** for token streaming.
- Tokens are sent immediately to client.
- Enhances user experience (real-time response).
- Works with OpenAI, Ollama, vLLM, etc.
- Can use SSE or plain text streaming.

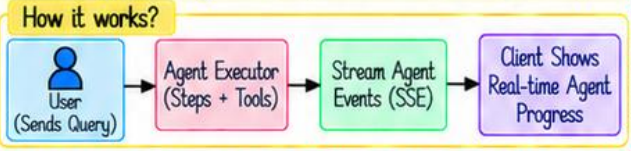
### Sample Output (Client View)

```
data: Hello
data: !
data: How
data: can
data: I ...
```

## 10 Agent Event Streaming

- Agent event streaming sends real-time updates of an AI agent's execution (thoughts, tool calls, observations, final answer).
- Helps users see what the agent is doing step-by-step.
- Useful for multi-step agent workflows, long-running tasks, and debugging.
- Implement using SSE and yield different event types (e.g., thinking, tool_call, observation, final_answer).

### How it works?




### Example: Agent Event Streaming (LangChain / LangGraph Style)

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import asyncio

app = FastAPI()

async def run_agent(query: str):
    # Simulated agent steps (replace with real agent logic)
    yield "event: thinking\data: Thinking about the query...\n\n"
    await asyncio.sleep(1)
    yield "event: tool_call\data: Searching knowledge base...\n\n"
    await asyncio.sleep(1)
    yield "event: observation\data: Found 3 relevant documents\n\n"
    await asyncio.sleep(1)
    yield "event: final_answer\data: Here is the answer: ...\n\n"

@app.get("/agent/stream")
async def agent_stream(query: str):
    return StreamingResponse(
        run_agent(query),
        media_type="text/event-stream"
    )
```

### Key Points:

- Streams agent's internal steps in real time.
- Event types: thinking, tool_call, observation, final_answer.
- Useful for complex agent workflows and debugging.
- Can be implemented with LangChain, LangGraph, or custom agents.

### Client (Browser) Example

```javascript
const eventSource = new EventSource('/agent/stream?query=RAG');

eventSource.addEventListener('thinking',
 (e) => console.log('Thinking:', e.data));
eventSource.addEventListener('tool_call',
 (e) => console.log('Tool Call:', e.data));
```

# Database Integration (Part 1)

PostgreSQL Connection • SQLAlchemy • Database Models



TheAgenticAIEngineer

## 1 PostgreSQL Connection

- PostgreSQL is a powerful open-source relational database.
- We use SQLAlchemy to connect and work with PostgreSQL in FastAPI.
- Connection string contains: username, password, host, port, database.
- Use environment variables to keep credentials secure.

### Database Connection URL Format

```
postgresql+psycopg2://username:password@host:port/dbname
```

### Example: Connection Setup (database.py)

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base
from dotenv import load_dotenv
import os

load_dotenv()
DATABASE_URL = os.getenv("DATABASE_URL")

# Create engine
engine = create_engine(
    DATABASE_URL,
    echo=True,     # Set False in production
    pool_pre_ping=True
)

# SessionLocal for database sessions
SessionLocal = sessionmaker(autocommit=False,
                            autoflush=False, bind=engine)

# Base class for models
Base = declarative_base()
```

### Key Points:

- Use psycopg2 driver for PostgreSQL.
- Use environment variables (.env).
- create_engine() creates a connection engine.
- sessionmaker() creates DB sessions.
- declarative_base() is the base class for models.
- pool_pre_ping=True helps avoid stale connections.

### Requirements (install)

```
pip install
fastapi
uvicorn
sqlalchemy
psycopg2-binary
python-dotenv
```

## 2 SQLAlchemy

- SQLAlchemy is a Python ORM (Object Relational Mapper).
- Helps us work with databases using Python classes instead of raw SQL.
- Provides engine, session, models, and queries.
- Works well with FastAPI for database operations.

### How SQLAlchemy Works?

Python Code (Models) → SQLAlchemy ORM → Generates SQL Queries → PostgreSQL

### Example: SQLAlchemy Session (get_db)

```python
from sqlalchemy.orm import Session
from .database import SessionLocal

# Dependency to get DB session
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

### Key Points:

- Session is used to interact with the database.
- Use dependency injection in FastAPI.
- Always close the session after request.
- Use try-finally to handle exceptions.

### Using in FastAPI Endpoint

```python
from fastapi import Depends, FastAPI
from sqlalchemy.orm import Session
from .database import get_db

app = FastAPI()

@app.get("/")
def read_root(db: Session = Depends(get_db)):
    return {"message": "Database connected!"}
```

## 3 Database Models

- Models are Python classes that represent database tables.
- We use SQLAlchemy ORM models with declarative_base().
- Each class maps to a table, and each attribute is a column.
- Models help us perform CRUD operations easily.

### Example: User Model (models.py)

```python
from sqlalchemy import Column, Integer, String,
                       Boolean, DateTime, func
from .database import Base

class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(100), nullable=False)
    email = Column(String(100), unique=True, index=True)
    is_active = Column(Boolean, default=True)
    created_at = Column(DateTime, default=func.now())
```

### Key Points:

- __tablename__ sets the table name.
- Column() defines column type and constraints.
- primary_key=True makes it a primary key.
- index=True improves query performance.
- unique=True ensures unique values.
- func.now() sets timestamp automatically.

### Common Column Types

| | |
|---|---|
| Integer | - Whole number |
| String | - Text data |
| Boolean | - True/False |
| DateTime | - Date and time |
| Float | - Decimal numbers |
| Text | - Large text |
| JSON | - JSON data |
| ForeignKey | - Reference to another table |

### Example: Create Tables

```python
from .database import engine, Base

Base.metadata.create_all(bind=engine)
# This creates all tables in the database
```

### Complete Flow

FastAPI App → API Endpoint (Depends on get_db) → SQLAlchemy Session → Database Models (Python Classes) → SQL Queries (ORM) → PostgreSQL Database

# Database Integration (Part 2)

Database Sessions • CRUD Operations

## 1 Database Sessions

- A database session is a connection to the database used to perform operations.
- In FastAPI, we use dependency injection to provide a session to each request.
- A session handles transactions (commit/rollback).
- Each request gets its own session, which is closed after the request.
- Using sessions prevents connection leaks and ensures data consistency.

### Example: Get DB Session (database.py)

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker
from sqlalchemy.orm import Session
from .models import Base
import os

# Create engine
engine = create_engine(os.getenv("DATABASE_URL"))

# Create session factory
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

# Dependency to get DB session
def get_db() -> Session:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

**Key Points:**

- Uses sessionmaker() to create sessions.
- get_db() is used as a dependency in FastAPI.
- commit() to save changes.
- rollback() in case of error.
- Each request gets a new session.
- Always close the session after use.

## 2 CRUD Operations

- CRUD stands for Create, Read, Update, Delete.
- We use SQLAlchemy ORM to perform CRUD operations on database models.
- CRUD operations are used to interact with database tables. (e.g., users, conversations, messages)
- We perform operations using a database session.
- After making changes, we use commit() to save data or rollback() to undo changes.

### Example: CRUD Operations (user_crud.py)

```python
from sqlalchemy.orm import Session
from . inport models, schemas

# Create
def create_user(db: Session, user: schemas.UserCreate):
    db_user = models.User(**user.model_dump())
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user

# Read (Get by ID)
def get_user(db: Session, user_id: int):
    return db.query(models.User).filter(
        models.User.id == user_id
    ).first()

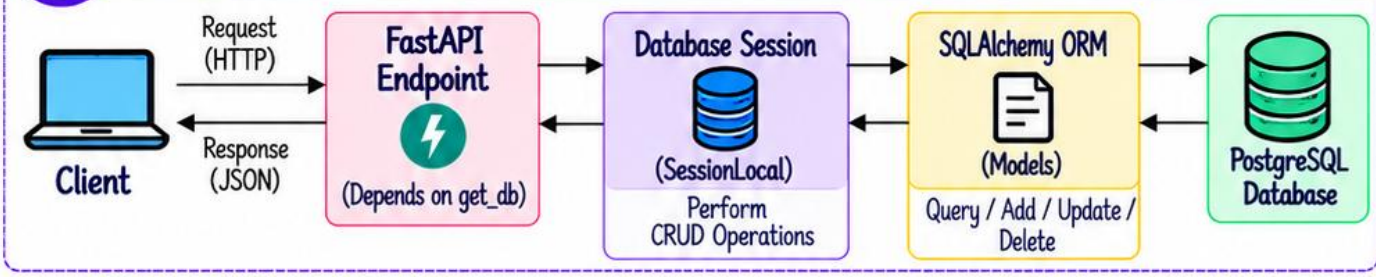
# Update
def update_user(db: Session, user_id: int, user_data):
    db_user = get_user(db, user_id)
    if db_user:
        for key, value in user_data.model_dump().items():
            setattr(db_user, key, value)
        db.commit()
        db.refresh(db_user)
    return db_user

# Delete
def delete_user(db: Session, user_id: int):
    db_user = get_user(db, user_id)
    if db_user:
        db.delete(db_user)
        db.commit()
    return db_user
```

**Key Points:**

- **Create**: Add new record using session.add() and commit().
- **Read**: Fetch records using query() and filter().
- **Update**: Modify existing record and commit().
- **Delete**: Remove record using delete() and commit().
- Use ORM models instead of raw SQL queries.
- Always handle errors and use rollback() when needed.

## 3 Complete Flow: FastAPI + Database (CRUD)



# Database Integration (Part 3)

Transactions • Connection Pooling • Alembic Migrations

## 1 Transactions

- A transaction is a set of database operations that are executed as a single unit.
- Either all operations succeed (commit) or all fail (rollback).
- Helps maintain data consistency.
- Very important when performing multiple related operations (e.g., create user + create conversation + create message).
- Handled using db.commit() and db.rollback().

### Example: Transaction (transaction.py)

```python
from sqlalchemy.orm import Session
from models import User, Conversation, Message

def create_user_with_conversation(
    db: Session, user_data, conv_data):
    try:
        user = User(**user_data)
        db.add(user)
        db.flush()    # to get user.id

        conv = Conversation(
            user_id=user.id, **conv_data
        )
        db.add(conv)
        db.commit()   # commit both operations
        return {"message": "Created successfully"}

    except Exception as e:
        db.rollback() # rollback on error
        raise e
```

### Key Points:

- Use db.commit() to save changes.
- Use db.rollback() to undo changes on error.
- Group related operations in a single transaction.
- Prevents partial data (e.g., user created but conversation failed).
- Can also use context manager with session.begin().

## 2 Connection Pooling

- Connection pooling maintains a pool of reusable database connections.
- Improves performance by reusing existing connections.
- Reduces the overhead of creating a new connection for every request.
- Handled automatically by SQLAlchemy using QueuePool (default).
- We can configure pool size, max overflow, timeout, etc.

### Example: Connection Pooling (database.py)

```python
from sqlalchemy import create_engine
from aqlalchemy.orm import sessionmaker

DATABASE_URL = "postgresql+psycopg2://user:pass@host:5432/db"

engine = create_engine(
    DATABASE_URL,
    pool_size=10,         # number of connections in pool
    max_overflow=20,      # extra connections if needed
    pool_timeout=30,      # wait time for connection (seconds)
    pool_recycle=1800,    # recycle connections after 30 min
    echo=False
)

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)
```

### Key Points:

- Uses QueuePool (default in SQLAlchemy).
- Configure pool_size, max_overflow, pool_timeout, pool_recycle.
- Helps handle high traffic and concurrent requests.
- Prevents connection exhaustion.
- No need to manage connections manually.

## 3 Alembic Migrations

- Alembic is a database migration tool for SQLAlchemy.
- Helps manage changes to database schema (tables, columns, indexes, etc.).
- Allows version control for schema changes.
- Used in real projects to update database schema safely without losing data.
- Common commands: init, revision, upgrade, downgrade.

### Example: Alembic Commands

```
# 1. Initialize Alembic
alembic init alembic

# 2. Configure database URL in alembic.ini
sqlalchemy.url = postgresql+psycopg2://user:pass@host:5432/db

# 3. Create a new migration (after model changes)
alembic revision --autogenerate -m "Add users table"

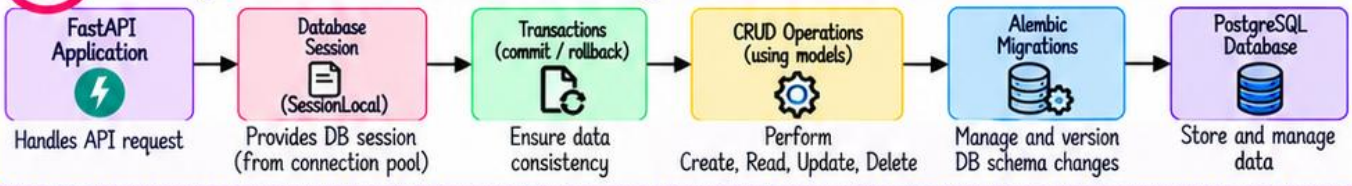
# 4. Apply migrations
alembic upgrade head

# 5. Rollback (if needed)
alembic downgrade -1
```

### Key Points:

- Tracks database schema changes.
- Autogenerates migration scripts from model changes.
- You can also write manual migration scripts.
- Use alembic upgrade head to apply changes.
- Use alembic downgrade to rollback.
- Essential for team projects and production deployments.

## 4 Complete Flow: Database Integration (Part 3)



# FastAPI + RAG + Agentic AI (Part 1)

## 1 RAG API Endpoint

- RAG API endpoint receives a user query and returns an answer using retrieved documents.
- Combines Retrieval (find relevant docs) + Generation (LLM creates answer).
- Provides a single endpoint for the complete RAG pipeline (search + LLM response).
- Can be extended later for Agentic AI.

### How it works?




### Example: RAG API Endpoint in FastAPI

```python
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()
class QueryRequest(BaseModel):
    query: str
    top_k: int = 5
class QueryResponse(BaseModel):
    answer: str
    sources: list[str]
@app.post("/rag")
async def rag_endpoint(req: QueryRequest) -> QueryResponse:
    # 1. Retrieve relevant documents
    docs = retrieve_documents(req.query, top_k=req.top_k)
    # 2. Generate answer using LLM with retrieved context
    answer, sources = generate_rag_answer(req.query, docs)
    return QueryResponse(answer=answer, sources=sources)
```

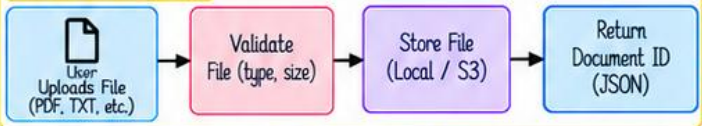
### Key Points:

- Uses request/response schemas (Pydantic).
- Retrieves relevant documents from vector DB.
- Passes context to LLM.
- Returns answer with sources.
- Can be part of larger Agentic AI system.

## 2 Document Upload API

- Allows users to upload documents (PDF, TXT, DOCX, etc.).
- Validates file type, size and stores the file in local storage or cloud storage (e.g., S3).
- Returns a document ID for further processing (ingestion).
- Can support single or multiple file uploads.

### How it works?




### Example: Document Upload API in FastAPI

```python
from fastapi import FastAPI, UploadFile, File, Depends
from pydantic import BaseModel
import uuid, os
app = FastAPI()
class UploadResponse(BaseModel):
    document_id: str
    filename: str
    message: str
UPLOAD_DIR = "./data/uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)
@app.post("/upload", response_model=UploadResponse)
async def upload_document(file: UploadFile = File(...)):
    # validate file type
    allowed_types = ["application/pdf", "text/plain",
    "application/vnd.openxmlformats-officedocument.wordprocessingml.document"]
    if file.content_type not in allowed_types:
        raise HTTPException(status_code=400, detail="File type not allowed")
    # save file
    doc_id = str(uuid.uuid4())
    file_path = os.path.join(UPLOAD_DIR, f"{doc_id}_{file.filename}")
    with open(file_path, "wb") as f:
        content = await file.read()
        f.write(content)
    return UploadResponse(document_id=doc_id, filename=file.filename,
            message="File uploaded successfully")
```

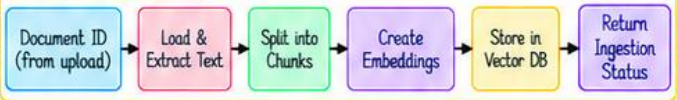
### Key Points:

- Supports multiple file types.
- Validates file before upload.
- Stores file locally or in cloud.
- Returns a unique document ID.
- Can be extended with authentication, file size limits, and virus scanning.

## 3 Document Ingestion API

- Ingestion API processes uploaded documents.
- It loads the file, splits it into chunks, creates embeddings and stores them in a vector database (e.g., FAISS, ChromaDB).
- Returns ingestion status and metadata (number of chunks, etc.).
- Can run in background for large files (using async tasks).

### How it works?




### Example: Document Ingestion API in FastAPI

```python
from fastapi import FastAPI, BackgroundTasks
from pydantic import BaseModel
app = FastAPI()
class IngestRequest(BaseModel):
    document_id: str
    chunk_size: int = 500
    chunk_overlap: int = 50
class IngestResponse(BaseModel):
    document_id: str
    chunks_created: int
    status: str
@app.post("/ingest", response_model=IngestResponse)
async def ingest_document(req: IngestRequest, background_tasks: BackgroundTasks):
    # run ingestion in background
    background_tasks.add_task(process_document, req.document_id,
        req.chunk_size, req.chunk_overlap)
    return IngestResponse(document_id=req.document_id,
        chunks_created=0, status="Ingestion started")
```

### Key Points:

- Processes uploaded document.
- Text extraction + chunking.
- Creates embeddings using Sentence Transformers or similar model.
- Stores embeddings in vector DB.
- Can run as background task for large documents.
- Returns ingestion status.

## 4 Search / Retrieval API

- Search API retrieves relevant document chunks for a given query.
- Uses vector similarity search (and optionally hybrid search with keyword/BM25).
- Returns top-k relevant chunks with metadata (source, page, score).
- Used by RAG and Agentic AI to provide context to LLM.

### How it works?




### Example: Search / Retrieval API in FastAPI

```python
from fastapi import FastAPI
from pydantic import BaseModel
app = FastAPI()
class SearchRequest(BaseModel):
    query: str
    top_k: int = 5
class SearchResult(BaseModel):
    text: str
    source: str
    page: int | None = None
    score: float
@app.post("/search", response_model=list[SearchResult])
async def search_documents(req: SearchRequest):
    # create query embedding
    query_embedding = create_embedding(req.query)
    # search vector DB
    results = vector_search(query_embedding, top_k=req.top_k)
    return [SearchResult(**r) for r in results]
```

### Key Points:

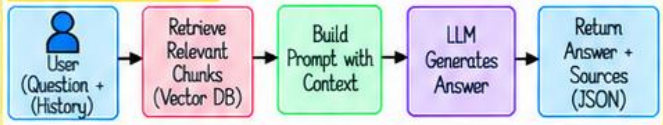
- Uses vector similarity search.
- Can combine with keyword search (hybrid retrieval).
- Returns relevant chunks with metadata and similarity score.
- Used by RAG endpoint to build context for LLM.
- Supports pagination, filters (e.g., document_id, date, etc.).

# FastAPI + RAG + Agentic AI (Part 2)

## 5 Chat API (RAG + LLM)

- Chat API receives user question and returns a grounded answer using RAG (retrieve → augment → generate).
- Uses conversation history (optional).
- Returns answer with sources (document references).
- Can be extended to support streaming (SSE) later.
- Used for a production chat interface (like JioAI, Onyx).

### How it works?




### Example: Chat API in FastAPI (RAG)

```python
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    question: str
    chat_history: list[dict] = []
    top_k: int = 5

class ChatResponse(BaseModel):
    answer: str
    sources: list[dict]

@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(req: ChatRequest):
    # 1. Retrieve relevant documents
    docs = retriever.get_relevant_documents(req.question, k=req.top_k)
    # 2. Build prompt with context
    prompt = build_rag_prompt(req.question, docs, req.chat_history)
    # 3. Generate answer using LLM
    answer = llm.invoke(prompt)
    # 4. Return answer + sources
    return ChatResponse(
        answer=answer.content,
        sources=[{"page": d.metadata.get("page"),
                  "source": d.metadata.get("source")}
                 for d in docs]
)
```

### Key Points:

- Uses Pydantic request/response models.
- Retrieves top-k chunks.
- Builds context-aware prompt.
- Returns answer with sources.
- Can store and use chat history.
- Can be extended for streaming response.

## 6 Agent Execution API

- Agent Execution API runs an AI agent to complete a user goal.
- Agent can plan, use tools (search, DB, API, etc.), and reason.
- Returns final answer along with intermediate steps.
- Useful for complex tasks (multi-step, tool calling, reason & act).
- Can be built using LangChain agents or LangGraph agents.

### How it works?




### Example: Agent Execution API (LangChain Agent)

```python
from fastapi import FastAPI
from pydantic import BaseModel
from langchain.agents import initialize_agent, AgentType

app = FastAPI()

class AgentRequest(BaseModel):
    task: str
    chat_history: list[dict] = []

class AgentResponse(BaseModel):
    answer: str
    steps: list[dict]

@app.post("/agent/run", response_model=AgentResponse)
async def run_agent(req: AgentRequest):
    agent_executor = initialize_agent(
        tools=tools,
        llm=llm,
        agent=AgentType.OPENAI_FUNCTIONS,
        verbose=True,
        return_intermediate_steps=True
    )
    result = await agent_executor.ainvoke({
        "input": req.task,
        "chat_history": req.chat_history
    })
    return AgentResponse(
        answer=result["output"],
        steps=result.get("intermediate_steps", [])
    )
```

### Key Points:

- Takes a user task/goal.
- Agent decides steps and uses tools.
- Returns final answer with intermediate steps.
- Supports multi-step reasoning and tool calling.
- Can use LangChain or LangGraph agents.

### Sample Response

```
{
 "answer": "The revenue last
   quarter was ₹2.5 Cr",
 "steps": [
   {"tool": "sql_query",
     "input": "...",
     "output": "..."},
   {"tool": "summary",
     "input": "...",
     "output": "..."}
   ]
}
```

## 7 FastAPI + LangGraph

- Use FastAPI to expose a LangGraph agent as an API.
- LangGraph helps build stateful, multi-step agent workflows with better control, memory, and tool usage.
- Agents can handle complex tasks with planning, tool calling, and human-in-the-loop.
- Used for production-grade Agentic AI applications.

### How it works?




### Example: FastAPI + LangGraph

```python
from fastapi import FastAPI
from pydantic import BaseModel
from langgraph.graph import StateGraph, END

app = FastAPI()

# Define state
class AgentState(BaseModel):
    query: str
    messages: list[dict] = []

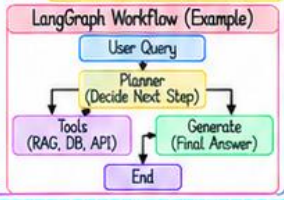
# Build LangGraph workflow (simplified)
workflow = StateGraph(AgentState)
workflow.add_node("planner", planner_node)
workflow.add_node("tools", tool_node)
workflow.add_node("generate", generate_node)
workflow.set_entry_point("planner")
workflow.add_edge("planner", "tools")
workflow.add_edge("tools", "generate")
workflow.add_edge("generate", END)
app_graph = workflow.compile()

@app.post("/langgraph/run")
async def run_langgraph(state: AgentState):
    result = await app_graph.ainvoke(state.dict())
    return {"answer": result["answer"], "result": "progress:
                                                  "messages"}
```

### Key Points:

- Uses LangGraph for stateful agent workflows.
- Supports multi-agent flows, conditional routing, memory.
- Better for complex, real-world agentic tasks.
- Can stream intermediate events to client (SSE).
- Easily integrates with tools (RAG, DB, APIs).

### LangGraph Workflow (Example)




## 8 FastAPI + LangChain

- Use FastAPI to expose LangChain chains, tools, and agents as APIs.
- LangChain provides abstractions for RAG, tools, memory, and agents.
- Can build simple chains (RAG) or advanced agents (tool calling).
- Works well with vector databases, LLMs, and external APIs.
- Easy to extend and integrate with existing systems.

### How it works?




### Example: FastAPI + LangChain (RAG Chain)

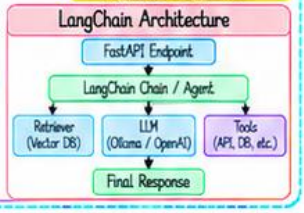
```python
from fastapi import FastAPI
from pydantic import BaseModel
from langchain.chains import RetrievalQA
from langchain_community.chat_models import ChatOllama
app = FastAPI()
# Initialize RAG chain
retriever = vector_store.as_retriever(search_kwargs={"k": 5})
llm = ChatOllama(model="llama3", temperature=0)
rag_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True
)
class QueryRequest(BaseModel):
    question: str

@app.post("/rag", response_model=dict)
async def rag_endpoint(req: QueryRequest):
    result = rag_chain.invoke({"query": req.question})
    return {
        "answer": result["result"],
        "sources": [doc.metadata for doc in
                    result["source_documents"]]
    }
```

### Key Points:

- Supports RAG chains, tools, and agents.
- Easy integration with vector DBs and LLMs.
- Can use LangChain agents with tools (search, DB, API).
- Simple and modular code.
- Widely used for building production RAG and Agentic AI systems.

### LangChain Architecture

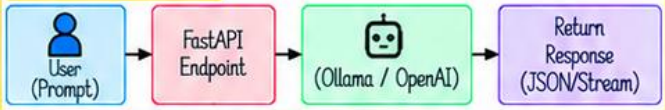


# FastAPI + RAG + Agentic AI (Part 3)

## 9 FastAPI + LLM

- Expose LLM as an API using FastAPI.
- Supports normal and streaming response.
- Can use local LLM (Ollama) or external LLM (OpenAI etc.).
- Used for chat, summarization, Q&A, etc.

### How it works?




### Example: LLM API with Streaming (Ollama)

```python
from fastapi import FastAPI
from fastapi.responses import StreamingResponse
import requests, json

app = FastAPI()

@app.post("/llm/stream")
async def stream_llm(prompt: str):
    def generate():
        resp = requests.post(
            "http://localhost:11434/api/generate",
            json={"model": "llama3", "prompt": prompt, "stream": True},
            stream=True
        )
        for line in resp.iter_lines():
            if line:
                data = json.loads(line)
                token = data.get("response", "")
                if token:
                    yield f"data: {token}\n\n"

    return StreamingResponse(generate(), media_type="text/event-stream")
```

**Key Points:**

- Can use Ollama, OpenAI, Gemini, etc.
- Supports streaming tokens.
- Can add temperature, max_tokens, etc.
- Useful for chat, RAG, agentic applications.

## 10 FastAPI + Vector DB

- Use FastAPI to connect with a vector database (FAISS, ChromaDB, etc.).
- Store document embeddings.
- Search similar documents using embeddings.
- Used for RAG retrieval.

### How it works?




### Example: Vector Search API (FAISS)

```python
from fastapi import FastAPI
from pydantic import BaseModel
from sentence_transformers import SentenceTransformer
import faiss, numpy as np

app = FastAPI()

class SearchRequest(BaseModel):
    query: str
    top_k: int = 5

model = SentenceTransformer("BAAI/bge-base-en")
index = faiss.read_index("data/faiss.index")
documents = np.load("data/docs.npy", allow_pickle=True)

@app.post("/search")
async def search(req: SearchRequest):
    query_vec = model.encode([req.query])
    distances, indices = index.search(query_vec, req.top_k)
    results = []
    for idx, score in zip(indices[0], distances[0]):
        results.append({
            "id": int(idx),
            "content": documents[idx].item(),
            "score": float(score)
        })
    return {"results": results}
```

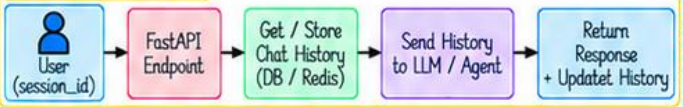
**Key Points:**

- Use embeddings to convert query to vector.
- Search top-k similar documents.
- Can use FAISS, ChromaDB, Pinecone, Weaviate, etc.
- Return document content and metadata.

## 11 Conversation / Session Handling

- Maintain conversation history for multi-turn chat.
- Use session_id (from user) to store messages.
- Store in memory (Redis) or database (PostgreSQL, SQLite).
- Retrieve history and pass to LLM/Agent for contextual answers.

### How it works?




### Example: Session-based Chat (Redis)

```python
from fastapi import FastAPI, Depends
from pydantic import BaseModel
import redis, json

app = FastAPI()
r = redis.Redis(host="localhost", port=6379,
 decode_responses=True)

class ChatRequest(BaseModel):
    session_id: str
    message: str

@app.post("/chat")
async def chat(req: ChatRequest):
    key = f"session:{req.session_id}"
    history = json.loads(r.get(key) or "[]")
    # add user message
    history.append({"role": "user", "content": req.message})
    # call LLM (dummy example)
    answer = "This is a response from LLM with context."
    history.append({"role": "asiistant", "content": answer})
    # save updated history
    r.set(key, json.dumps(history))
    return {"answer": answer, "history": history}
```

**Key Points:**

- Use session_id to track user conversations.
- Store history in Redis/ PostgreSQL/SQLite.
- Pass relevant history to LLM for contextual responses.
- Can set TTL for old sessions.
- Useful for Chatbots, RAG, and Agentic AI.

## 12 Tool / API Integration

- Agents can use external tools and APIs to get real-time information.
- Use FastAPI endpoints to integrate tools (web search, calculator, weather API, database queries, etc.).
- Can be used with LangChain or LangGraph tool calling.
- Supports synchronous and asynchronous tool execution.

### How it works?




### Example: Tool/API Integration (Weather Tool)

```python
from fastapi import FastAPI
from pydantic import BaseModel
import httpx
app = FastAPI()

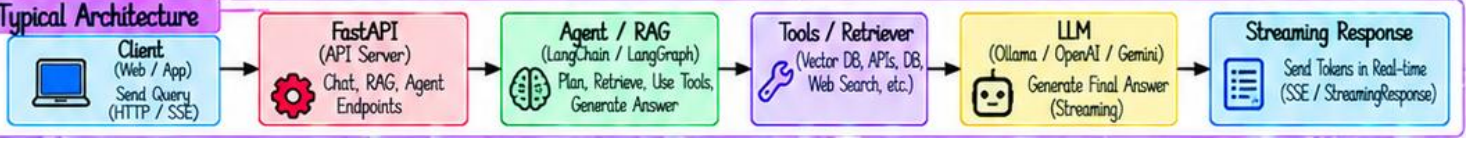
class WeatherRequest(BaseModel):
    city: str

@app.get("/tools/weather")
async def get_weather(city: str):
    url = f"https://api.open-meteo.com/v1/forecast?"\
          f"city={city}&current_weather=true"
    async with httpx.AsyncClient() as client:
        resp = await client.get(url)
        data = resp.json()
    return {"city": city, "weather": data}
```

**Key Points:**

- Integrate real-time APIs (web search, weather, etc.).
- Agents decide when to use a tool based on user query.
- Can define tools in LangChain or LangGraph.
- Supports authentication, error handling, and retries.
- Can add custom tools (database query, internal APIs, file operations, etc.).

### Typical Architecture



# Error Handling & Testing (Part 1)

## Error Handling in FastAPI (Part 1)

Error handling helps your API return meaningful errors, handle unexpected issues, and improve user experience.

## 1 HTTPException

- FastAPI provides HTTPException to raise HTTP errors with status code and detail message.
- Used for expected errors (e.g., resource not found, unauthorized access).
- Automatically returns a JSON error response.

### Example

```python
from fastapi import FastAPI, HTTPException
app = FastAPI()

users = {1: {"id": 1, "name": "Ali"}}

@app.get("/users/{user_id}")
def get_user(user_id: int):
    if user_id not in users:
        raise HTTPException(
            status_code=404,
            detail=f"User with id {user_id} not found"
    )
    return users[user_id]
```

### Key Points:

- Raise HTTPException with status_code and detail.
- Common status codes: 400 (Bad Request), 401 (Unauthorized), 403 (Forbidden), 404 (Not Found), 500 (Internal Error).
- FastAPI returns standard JSON format.

### Response (Error)

```
{
    "detail": "User with id 2 not found"
}
```

Status Code: 404

## 2 Custom Exceptions

- Create custom exception classes for application-specific errors.
- Helps maintain clean and reusable error handling logic.
- Useful for business logic errors (e.g., invalid input, insufficient credits, etc.).

### Example

```python
from fastapi import FastAPI, HTTPException
app = FastAPI()

# 1. Create custom exception
class InsufficientCreditsError(Exception):
    def __init__(self, message: str = "Not enough credits"):
        self.message = message

# 2. Raise custom exception
@app.post("/generate")
def generate(content: str, credits: int):
    if credits < 10:
        raise InsufficientCreditsError(
                "You need at least 10 credits"
        )
    return {"message": "Content generated successfully"}
```

### Key Points:

- Create custom exception by inheriting from Exception.
- Use for business logic or domain-specific errors.
- Handle it using exception handlers (see next section).

### Raised Exception

```python
raise InsufficientCreditsError("You need at least 10 credits")
```

### Handled Response

```
{
    "detail": "You need at least 10 credits"
}
```

## 3 Exception Handlers

- Use exception handlers to customize error responses.
- Can handle custom exceptions, HTTPException, or any unhandled exceptions.
- Helps return consistent error format across the application.

### Example

```python
from fastapi import FastAPI, Request, HTTPException
from fastapi.responses import JSONResponse
app = FastAPI()

# Custom exception
class InsufficientCreditsError(Exception):
    def __init__(self, message: str):
        self.message = message

# Exception handler for custom exception
@app.exception_handler(InsufficientCreditsError)
async def insufficient_credits_handler(request: Request,
    exc: InsufficientCreditsError):
    return JSONResponse(
        status_code=400,
        content={
            "error": "INSUFFICIENT_CREDITS",
            "message": exc.message
        }
    )

@app.get("/test")
def test():
    raise InsufficientCreditsError("You need at least 10 credits")
```

### Key Points:

- Use @app.exception_handler(ExceptionClass) to handle specific exceptions.
- Return custom JSON response using JSONResponse.
- Can also handle generic Exception to catch unexpected errors.

### Response (Custom Format)

```
{
    "error": "INSUFFICIENT_CREDITS",
    "message": "You need at least 10 credits"
}
```

Status Code: 400

## 4 Validation Errors

- FastAPI automatically validates request data using Pydantic.
- If the input data is invalid, it returns a 422 Unprocessable Entity error with detailed information.
- You can customize validation error responses using exception handlers.

### Example

```python
from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field
app = FastAPI()

class UserRequest(BaseModel):
    name: str = Field(..., min_length=3)
    age: int = Field(..., ge=18)

# Custom handler for validation errors
@app.exception_handler(RequestValidationError)
async def validation_exception_handler(
    request: Request, exc: RequestValidationError
):
    return JSONResponse(
        status_code=422,
        content={
            "error": "VALIDATION_ERROR",
            "message": "Invalid request data",
            "details": exc.errors()
        }
    )
@app.post("/users")
def create_user(user: UserRequest):
    return {"message": "User created", "data": user}
```

### Key Points:

- FastAPI uses Pydantic for automatic request validation.
- Validation errors return 422 status code.
- exc.errors() provides detailed field errors (location, message, error type).
- You can customize the response format using @app.exception_handler (RequestValidationError).

### Response (Validation Error)

```
{
    "error": "VALIDATION_ERROR",
    "message": "Invalid request data",
    "details": [
        {
            "loc": ["body", "name"],
            "msg": "String should have at least 3 characters",
            "type": "string_too_short",
            "input": "Al"
        }
    ]
}
```

Status Code: 422

## Error Handling Flow in FastAPI



# Error Handling & Testing (Part 2)

## 1 LLM / Tool Failures

- Handle failures from LLM calls (API errors, rate limits, invalid responses).
- Handle tool execution failures (API errors, network issues, invalid output).
- Implement retries, fallbacks and graceful error messages.
- Return meaningful error to the client (not raw stack trace).
- Useful for RAG and Agentic AI applications.

### Example: LLM Call with Retry and Fallback

```python
from fastapi import FastAPI, HTTPException
from openai import OpenAI
import time

app = FastAPI()
client = OpenAI(api_key="YOUR_API_KEY")

@app.get("/llm")
async def call_llm(prompt: str):
    max_retries = 3
    for attempt in range(max_retries):
        try:
            response = client.chat.completions.create(
                model="gpt-4o-mini",
                messages=[{"role": "user", "content": prompt}],
                timeout=20
            )
            return {"answer": response.choices[0].message.content}
        except Exception as e:
            if attempt == max_retries - 1:
                # fallback response
                return {
                    "error": "LLM service unavailable",
                    "message": "Please try again later"
                }
            time.sleep(2 ** attempt)  # exponential backoff
```

### Key Points:

- Use try-except to handle LLM errors.
- Implement retry with exponential backoff.
- Use fallback model or cached response.
- Return user-friendly error message.
- Log the full error for debugging.
- Handle rate limits, invalid responses, network errors, etc.
- Useful for production systems.

### Sample Error Response

```
{
  "error": "LLM service unavailable",
  "message": "Please try again later"
}
```

Status Code: 503

## 2 Timeout Handling

- Set timeouts for external API calls (LLM, tools, vector DB, etc.).
- Prevent long-running requests from blocking the server.
- Return a clear timeout error to the client.
- Can be combined with retries and fallback.
- Useful for reliable and responsive applications.

### Example: Timeout Handling for External API Call

```python
from fastapi import FastAPI, HTTPException
import httpx

app = FastAPI()

@app.get("/tool/search")
async def search_tool(query: str):
    url = "https://api.example.com/search"
    try:
        async with httpx.AsyncClient(timeout=10.0) as client:
            response = await client.get(url, params={"q": query})
            response.raise_for_status()
            return response.json()
    except httpx.TimeoutException:
        raise HTTPException(
            status_code=504,
            detail="Tool API request timed out. Please try again."
        )
    except httpx.HTTPStatusError as e:
        raise HTTPException(
            status_code=e.response.status_code,
            detail=f"Tool API error: {e.response.text}"
        )
```

### Key Points:

- Use timeout parameter in HTTP client.
- Handle TimeoutException separately.
- Return 504 (Gateway Timeout) for timeouts.
- Combine with retries or fallback.
- Set reasonable timeout (5–30s).
- Useful for LLM calls, tool APIs, vector DB, etc.

### Sample Timeout Response

```
{
  "detail": "Tool API request timed out. Please try again."
}
```

Status Code: 504

## 3 Testing with pytest

- pytest is a popular testing framework for Python.
- Used to write and run automated tests for FastAPI APIs.
- Helps ensure API endpoints work as expected.
- Can test success cases, error cases, and edge cases.
- Supports fixtures, parameterized tests, and async tests.

### Example: Basic pytest Test for FastAPI

```python
# file: test_main.py
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_root():
    response = client.get("/")
    assert response.status_code == 200
    assert response.json() == {"message": "Hello World"}

def test_chat_success():
    response = client.post(
        "/chat",
        json={"question": "What is AI?"}
    )
    assert response.status_code == 200
    assert "answer" in response.json()

def test_invalid_input():
    response = client.post(
        "/chat",
        json={"question": ""}
    )
    assert response.status_code == 422
```

### Key Points:

- Use TestClient to call FastAPI endpoints.
- Test success and error cases.
- Check status code, response JSON, and data.
- Can test multiple endpoints.
- Supports fixtures and async tests.
- Run tests with pytest command.

### Run Tests

```
# Run all tests
pytest

# Run with verbose output
pytest -v
# Run a specific test file
pytest test_main.py
```

### Sample Test Output

```
========== test session starts ===
collected 3 items

test_main.py::test_root PASSED
test_main.py::test_chat_success PASSED
test_main.py::test_invalid_input PASSED
========== 3 passed in 1.23s ===
```

## 4 API Testing with TestClient

- TestClient is provided by FastAPI (built on requests).
- Used to test API endpoints without running a live server.
- Can test GET, POST, PUT, DELETE requests.
- Can check headers, query params, request body, and response.
- Useful for unit testing and integration testing.

### Example: API Testing with TestClient

```python
from fastapi.testclient import TestClient
from main import app

client = TestClient(app)

def test_get_items():
    response = client.get("/items?category=books")
    assert response.status_code == 200
    data = response.json()
    assert isinatance(data, list)

def test_create_item():
    payload = {"name": "Laptop", "price": 50000}
    response = client.post("/items", json=payload)
    assert response.status_code == 201
    assert response.json()["name"] == "Laptop"

def test_not_found():
    response = client.get("/items/999")
    assert response.status_code == 404
    assert response.json()["detail"] == "Item not found"
```

### Key Points:

- Can test different HTTP methods (GET, POST, PUT, DELETE).
- Test query params, headers, and JSON body.
- Check response status code, data, and headers.
- Can test authentication, RBAC, and error cases.
- Useful for CI/CD pipelines.

### Sample API Test Cases

| Test Case | Method | Endpoint | Expected Status | Purpose |
|---|---|---|---|---|
| Get items | GET | /items | 200 | Test success response |
| Create item | POST | /items | 201 | Test create operation |
| Invalid input | POST | /items | 422 | Test validation error |
| Item not found | GET | /items/999 | 404 | Test error response |
| Unauthorized | GET | /admin | 401 | Test authentication |

# Error Handling & Testing (Part 3)

Async Testing, Mocking, Dependency Overrides, and Endpoint Testing for RAG & Agent

## 1 Async Testing in FastAPI

- Use pytest with asyncio for testing async endpoints.
- Test async functions and async API calls.
- Use httpx.AsyncClient for async API testing.
- Helpful for endpoints that call LLMs, vector DB, or other services.

### Example: Async Endpoint Test

```python
# test_async_endpoint.py
import pytest
from httpx import AsyncClient
from main import app

@pytest.mark.asyncio
async def test_rag_endpoint():
    async with AsyncClient(app=app,
            base_url="http://test") as ac:
        response = await ac.post("/rag",
            json={"query": "What is RAG?"})
        assert response.status_code == 200
        data = response.json()
        assert "answer" in data
        assert len(data["sources"]) > 0
```

**Key Points:**

- Use @pytest.mark.asyncio for async tests.
- Use httpx.AsyncClient instead of TestClient for async testing.
- Can test async endpoints, streaming endpoints, and background tasks.
- Works well for RAG and Agent endpoints that call external services.

### Sample Test Output

```
$ pytest -v test_async_endpoint.py
test_async_endpoint.py::test_rag_endpoint  PASSED  [100%]
1 passed in 1.23s
```

## 2 Mocking

- Use mocking to replace external dependencies (LLM, Vector DB, APIs).
- Helps test logic without calling real services.
- Use pytest-mock or unittest.mock.
- Useful for fast, reliable, and cost-effective testing.

### Example: Mocking LLM Call

```python
# test_mock_llm.py
import pytest
from httpx import AsyncClient
from unittest.mock import AsyncMock, patch
from main import app

@pytest.mark.asyncio
@patch("app.services.llm_service.chat")
async def test_llm_mock(mock_chat):
    # Mock LLM response
    mock_chat.return_value = AsyncMock(
        return_value="This is a mocked answer"
    )
    async with AsyncClient(app=app, base_url="http://test") as ac:
        response = await ac.post("/chat",
                    json={"message": "Hello"})
        assert response.status_code == 200
        data = response.json()
        assert data["answer"] == "This is a mocked answer"
        mock_chat.assert_called_once()
```

**Key Points:**

- Use patch to replace external functions.
- Can mock LLM, vector DB, APIs, or tools.
- Use AsyncMock for async functions.
- Verify function calls and arguments.
- Helps in unit testing without real API calls.

### Sample Test Output

```
$ pytest -v test_mock_llm.py
test_mock_llm.py::test_llm_mock PASSED  [100%]
1 passed in 0.85s
```

## 3 Dependency Overrides

- FastAPI allows overriding dependencies in tests.
- Use to replace real services (LLM, DB, vector DB, tools) with mocks.
- Helps test endpoints in isolation.
- Commonly used for RAG and Agent endpoints.

### Example: Override Dependencies in Test

```python
# test_dependency_override.py
import pytest
from httpx import AsyncClient
from main import app
from app.dependencies import get_llm, get_vector_db
from app.services.mock_services import MockLLM, MockVectorDB

# Override dependencies
app.dependency_overrides[get_llm] = lambda: MockLLM()
app.dependency_overrides[get_vector_db] =
                        lambda: MockVectorDB()

@pytest.mark.asyncio
async def test_rag_with_override():
    async with AsyncClient(app=app,base_url="http://test") as ac:
        response = await ac.post("/rag",json={"query": "RAG!"})
        assert response.status_code == 200
        data = response.json()
        assert "answer" in data

# Clear overrides after test
app.dependency_overrides.clear()
```

**Key Points:**

- Use app.dependency_overrides to replace dependencies.
- Can override LLM, vector DB, tools, or database.
- Useful for integration testing.
- Helps make tests independent and repeatable.

### Dependency Override Flow




## 4 RAG & Agent Endpoint Testing

- Test RAG endpoint: document retrieval, context building, final answer.
- Test Agent endpoint: multi-step execution, tool calling, and final response.
- Use mocking and dependency overrides for external services.
- Test success cases, error cases, and edge cases.

### Example: RAG Endpoint Test

```python
# test_rag_endpoint.py
import pytest
from httpx import AsyncClient
from main import app
from app.dependencies import get_retriever, get_llm
app.dependency_overrides[get_retriever] = lambda: MockRetriever()
app.dependency_overrides[get_llm] = lambda: MockLLM()

@pytest.mark.asyncio
async def test_rag_endpoint():
    async with AsyncClient(app=app, base_url="http://test") as ac:
        response = await ac.post("/rag", json={"query": "AI agents"})
        assert response.status_code == 200
        data = response.json()
        assert "answer" in data
        assert len(data["sources"]) > 0
```

**Key Points (RAG):**

- Mock retriever and LLM.
- Test retrieval, context building, and final answer.
- Check response format and sources.

### Example: Agent Endpoint Test

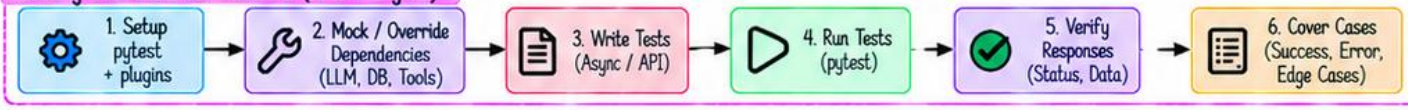
```python
# test_agent_endpoint.py
import pytest
from httpx import AsyncClient
from main import app
from app.dependencies import get_agent_executor
app.dependency_overrides[get_agent_executor] = lambda: MockAgent()

@pytest.mark.asyncio
async def test_agent_endpoint():
    async with AsyncClient(app=app,base_url="http://test") as ac:
        response = await ac.post("/agent",json={"task":"get weather"})
        assert response.status_code == 200
        data = response.json()
        assert "answer" in data
        assert "intermediate_steps" in data
```

**Key Points (Agent):**

- Mock agent executor and tools.
- Test multi-step execution.
- Check final answer and intermediate steps.
- Test tool calling logic.
- Verify error handling (tool failures, timeouts).

### Testing Workflow for FastAPI (RAG + Agent)



# Middleware & Observability (Part 1)

Middleware • Request Logging • Response Logging

## 1 Middleware

- Middleware runs for every request before it reaches the route and after the response is generated.
- Used for cross-cutting concerns like logging, authentication, CORS, error handling, etc.
- In FastAPI, we use `@app.middleware("http")` to create custom middleware.
- We can access and modify both the request and response objects.

### Example: Basic Middleware (middleware.py)

```python
from fastapi import FastAPI, Request
from starlette.middleware.base import BaseHTTPMiddleware

app = FastAPI()

@app.middleware("http")
async def custom_middleware(request: Request, call_next):
    # Code to run before the request is processed

    response = await call_next(request)

    # Code to run after the response is generated

    return response
```

### Key Points:

- Runs for every request.
- Can read/modify request and response.
- Useful for logging, auth, CORS, etc.
- Call next() to pass request to the next handler.

## 2 Request Logging

- Request logging captures important details about incoming requests.
- We log method, URL, headers, query parameters, and request body (if needed).
- Helps in debugging, monitoring and tracking user activity.
- We can use middleware to log request details before calling the route.

### Example: Request Logging Middleware

```python
from fastapi import FastAPI, Request
import logging
import json

app = FastAPI()

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

@app.middleware("http")
async def log_requests(request: Request, call_next):
    body = await request.body()
    logging.info(f"Request: {request.method} {request.url}")
    logging.info(f"Headers: {dict(request.headers)}")
    if body:
        logging.info(f"Body: {body.decode()}")
    response = await call_next(request)
    return response
```

### Key Points:

- Log method, URL, headers, query params.
- Log request body for debugging (avoid sensitive data).
- Use logging module.

## 3 Response Logging

- Response logging captures details about the response sent back to the client.
- We log status code, response headers and response body (if needed).
- Helps in debugging, monitoring API usage and performance.
- We log response after calling the route (using call_next(request)).

### Example: Response Logging Middleware

```python
from fastapi import FastAPI, Request, Response
import logging
app = FastAPI()

@app.middleware("http")
async def log_response(request: Request, call_next):
    response: Response = await call_next(request)
    logging.info(f"Response Status: {response.status_code}")
    logging.info(f"Response Headers: {dict(response.headers)}")
    # Read response body (careful with large responses)
    response_body = b""
    async for chunk in response.body_iterator:
        response_body += chunk
    logging.info(f"Response Body: {response_body.decode()}")
    return Response(content=response_body,
                    status_code=response.status_code,
                    headers=dict(response.headers))
```

### Key Points:

- Log status code, response headers and body.
- Read response body using response.body_iterator.
- Be careful with large responses and sensitive data.
- Useful for monitoring and troubleshooting.

## 4 Complete Flow: Middleware with Request & Response Logging



# Middleware's & Observability (Part 2)

Request ID • Correlation ID • Error Logging

## 1 Request ID

- Request ID is a unique identifier for each incoming request.
- It helps to trace and debug individual requests.
- We generate a UUID for every request and add it to the logs and response headers.
- We store it in the request state so it can be used across the application.

### Example: Request ID Middleware (request_id.py)

```python
from fastapi import FastAPI, Request
from starlette.middleware.base import BaseHTTPMiddleware
import uuid
import logging

logger = logging.getLogger("app")

class RequestIDMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request,
                       call_next):
        request_id = str(uuid.uuid4())
        request.state.request_id = request_id
        logger.info(f"[Request ID] {request_id} - "
                    f"{request.method} {request.url}")

        response = await call_next(request)
        response.headers["X-Request-ID"] = request_id
        return response
```

**Key Points:**

- Generates a unique ID using UUID.
- Stored in request.state.
- Added to response headers (X-Request-ID).
- Useful for tracing and debugging.

## 2 Correlation ID

- Correlation ID is used to trace a request across multiple services in a distributed system.
- It is passed in the request headers (e.g., X-Correlation-ID) or generated if not provided.
- The same correlation ID is used in logs, downstream API calls, and responses.
- Helps in end-to-end tracing in microservices and observability tools like OpenTelemetry, ELK, etc.

### Example: Correlation ID Middleware (correlation_id.py)

```python
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware
import uuid
import logging

logger = logging.getLogger("app")

class CorrelationIDMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request,
                       call_next):
        correlation_id = request.headers.get(
            "X-Correlation-ID", str(uuid.uuid4()))
        request.state.correlation_id = correlation_id
        logger.info(f"[Correlation ID] {correlation_id} - "
                    f"{request.method} {request.url}")

        response = await call_next(request)
        response.headers["X-Correlation-ID"] = correlation_id
        return response
```

**Key Points:**

- Uses existing X-Correlation-ID or generates new one.
- Stored in request.state.
- Added to response headers.
- Helps trace requests across multiple services.

## 3 Error Logging

- Error logging captures exceptions and logs detailed information.
- We log error type, message, request ID, correlation ID, stack trace, and request details.
- Helps in debugging and monitoring application issues.
- We can return a proper error response to the client without exposing sensitive details.

### Example: Error Logging Middleware (error_logging.py)

```python
from fastapi import Request
from starlette.middleware.base import BaseHTTPMiddleware
import logging
import traceback

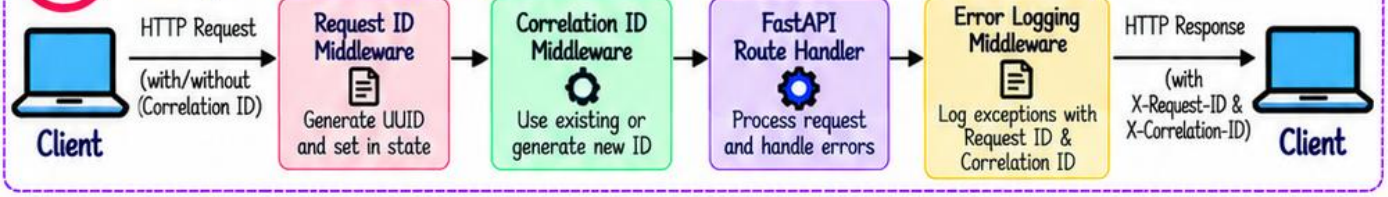
logger = logging.getLogger("app")

class ErrorLoggingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request,
                       call_next):
        try:
            return await call_next(request)
        except Exception as e:
            request_id = getattr(request.state, "request_id", "-")
            correlation_id = getattr(request.state, "correlation_id", "-")
            logger.error(
                f"[ERROR] req_id={request_id} corr_id={correlation_id} "
                f"path={request.url} - {str(e)}\n{traceback.format_exc()}"
            )
            return JSONResponse(status_code=500, content={"detail": "Internal Server Error"})
```

**Key Points:**

- Catches and logs all exceptions.
- Logs request ID, correlation ID, error message and stack trace.
- Helps in monitoring and debugging.
- Returns a safe error response to the client.

## 4 Complete Flow: Request → Middleware → API → Response



# Middleware & Observability (Part 3)

Request Timing • Health-check Endpoint

## 1 Request Timing

- Request timing measures how long each request takes to complete.
- We capture the start time before calling the route and calculate the total processing time after the response is generated.
- Helps in monitoring performance, identifying slow endpoints and debugging issues.
- We log the request path, method, status code and total time taken.

### Example: Request Timing Middleware (timing_middleware.py)

```python
from fastapi import FastAPI, Request
import time
import logging
from starlette.middleware.base import BaseHTTPMiddleware

logger = logging.getLogger("app")

class RequestTimingMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request: Request, call_next):

        start_time = time.time()
        response = await call_next(request)
        process_time = time.time() - start_time

        logger.info(
         f"{request.method} {request.url.path} "
         f"-{response.status_code} - {process_time:.3f}s"
        )
        response.headers["X-Process-Time"] = f"{process_time:.3f}"
        return response
```

**Key Points:**

- Uses time.time() to calculate request duration.
- Logs method, path, status code and time taken.
- Adds X-Process-Time header in response.
- Helps identify slow endpoints and performance bottlenecks.

## 2 Health-check Endpoint

- Health-check endpoint is used to check if the application is running and healthy.
- It is commonly used by load balancers, container orchestration tools (like Docker, Kubernetes) and monitoring systems.
- We can also check the health of dependent services like database connection.
- It should be lightweight and fast.

### Example: Health-check Endpoint (main.py)

```python
from fastapi import FastAPI, Depends
from sqlalchemy import text
from sqlaichemy.orm import Session

app = FastAPI()

# Database dependency (example)
def get_db() -> Session:
    ...

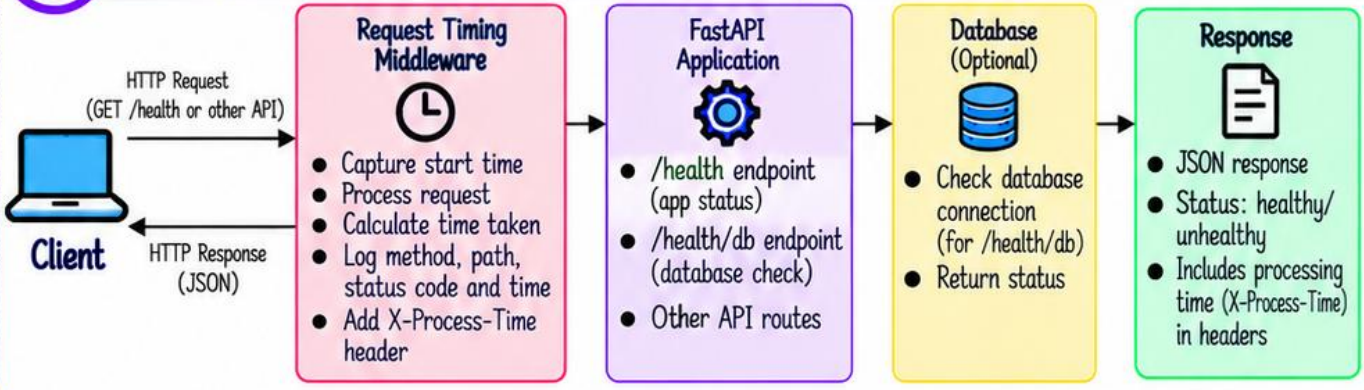
@app.get("/health")
async def health_check():
    return {"status": "healthy"}

@app.get("/health/db")
async def health_db(db: Session = Depends(get_db)):
    try:
      db.execute(text("SELECT 1"))
      return {"status": "healthy", "database": "ok"}
    except Exception as e:
      return {"status": "unhealthy",
              "database": "error",
              "detail": str(e)}
```

**Key Points:**

- /health → checks if the application is running.
- /health/db → checks database connectivity.
- Returns a simple JSON response (healthy/unhealthy).
- Useful for load balancers, container orchestration and monitoring tools.
- Should be fast, lightweight and not contain complex logic.

## 3 Complete Flow: Middleware with Request Timing & Health-check



# Middleware & Observability (Part 4)

OpenTelemetry Integration • LangSmith Integration

## 1 OpenTelemetry Integration

- OpenTelemetry (OTel) is an open-source standard for collecting and exporting traces, metrics and logs.
- Helps us observe the entire request flow across services (FastAPI, agents, tools, LLM calls).
- We can send data to backends like Jaeger, Grafana, or any OTLP-compatible service.
- Useful for monitoring performance, debugging and analysing latency in production.

## 2 LangSmith Integration

- LangSmith is an observability platform for LLM and agent applications.
- Helps us trace LLM calls, prompts, tool usage, agent steps and chain execution.
- Provides detailed logs, traces, token usage, cost tracking and evaluation.
- Useful for debugging, monitoring and improving RAG and Agentic AI applications.

### Example: OpenTelemetry Integration (main.py)

```python
from fastapi import FastAPI, Request
from opentelemetry import trace
from opentelemetry.instrumentation.fastapi
 import FastAPIInstrumetor
from opentelemetry.exporter.otlp.proto.http.
trace_exporter import OTLPSpanExporter
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import
 BatchSpanProcessor

app = FastAPI()

# Configure OpenTelemetry (send traces to
 Jaeger/OTLP endpoint)
trace.set_tracer_provider(TracerProvider())
span_processor = BatchSpanProcessor(
    OTLPSpanExporter(
    (endpoint="http://localhost:4318/v1/traces")
)
trace.get_tracer_provider().add_span_processor
 (span_processor)

# Instrument FastAPI automatically
FastAPIInstrumentor.instrument_app(app)

@app.get("/items")
async def read_items():
    return {"message": "Hello with OpenTelemetry!"}
```

**Key Points:**

- Uses OpenTelemetry SDK and FastAPI instrumentation.
- Sends traces to Jaeger or any OTLP endpoint.
- Automatically creates traces for every request.
- Helps track request flow, latency, errors and dependency calls.
- Useful for production monitoring and distributed tracing.

### Example: LangSmith Integration (main.py)

```python
from fastapi import FastAPI
from langsmith import Client
from langchain.callbacks.tracers import
     LangChainTracer
from langchain.callbacks.manager import
     CallbackManager

# Initialize LangSmith Client
client = Client(api_key="YOUR_LANGSMITH_API_KEY")

# Create tracer and callback manager
tracer = LangChainTracer(client=client)
callback_manager = CallbackManager([tracer])

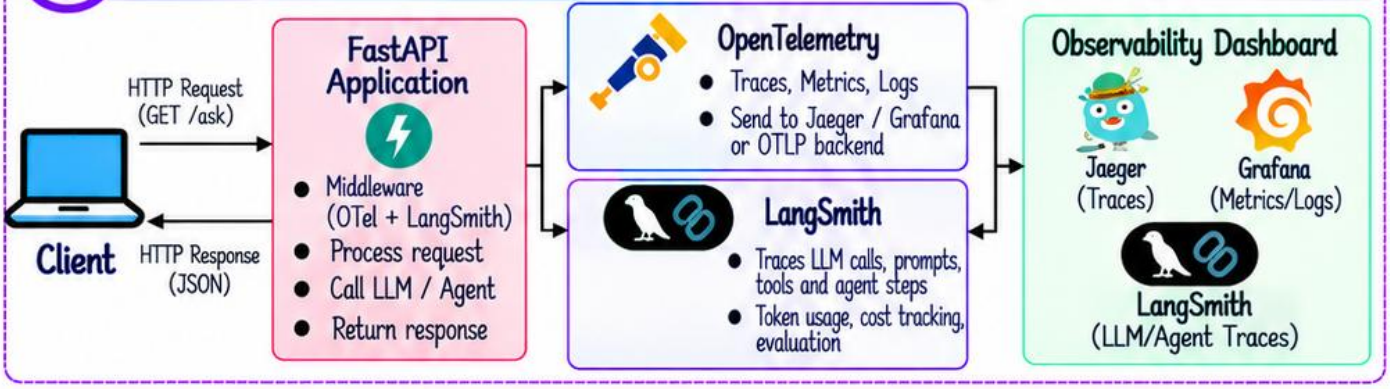
app = FastAPI()

@app.get("/ask")
async def ask_llm(question: str):
    # Example LLM call with LangSmith tracing
    from langchain.llms import OpenAI  # or any LLM
    llm = OpenAI(callbacks=callback_manager)
    response = llm.invoke(question)
    return {"answer": response}
```

**Key Points:**

- Uses LangSmith client and tracer for LLM/agent observability.
- Tracks prompts, LLM calls, tool usage and agent steps.
- Provides detailed traces, token usage and cost tracking.
- Helps in debugging, evaluation and improving AI applications.
- Widely used in RAG and Agentic AI projects.

## 3 Complete Flow: FastAPI with OpenTelemetry & LangSmith



# Creating FastAPI for Research AI Agents

## # Create FastAPI Backend

## 1 Project Structure

```
research_ai_agent/
├── main.py                 # FastAPI application
├── schemas.py              # Pydantic models
├── dependencies.py         # Dependencies (DB, etc.)
├── services/               # Business logic
│   └── research_service.py # Research service
├── agents/                 # AI agent logic
│   └── research_agent.py   # Research agent
└── .env                    # Environment variables
```

## 2 Install FastAPI

**Terminal**

```
pip install fastapi uvicorn
    pydantic python-dotenv
```

## 3 Create FastAPI Application

**main.py**

```python
from fastapi import FastAPI

app = FastAPI(
      title="Research AI Agent",
      version="1.0.0"
)

@app.get("/health")
async def health():
    return {"status": "healthy"}
```

**Run**

```
uvicorn main:app --reload
```

## 4 Create Research Request Schema

**schemas.py**

```python
from pydantic import BaseModel

class ResearchRequest(BaseModel):
      topic: str
      depth: str = "medium"
```

Pydantic automatically validates the input data sent by the client.

## 5 Create Research API

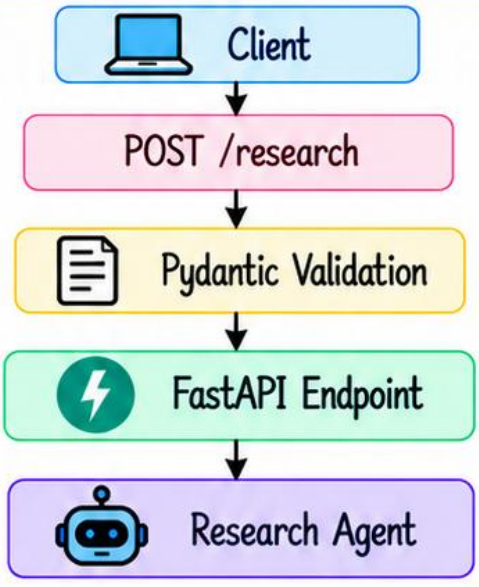
**main.py**

```python
from fastapi import FastAPI
from schemas import ResearchRequest

app = FastAPI()

@app.post("/research")
async def research(request: ResearchRequest):
    return {
        "topic": request.topic,
        "depth": request.depth,
        "status": "research started"
    }
```

## Flow



# Creating FastAPI for Research AI Agents

## # Connect FastAPI with Research Agent

## 6 Create Research Agent

agents/research_agent.py

```python
class ResearchAgent:
    async def run(self, topic: str, depth: str):
        # 1. Search web
        # 2. Collect sources
        # 3. Extract information
        # 4. Analyze information
        # 5. Generate report

        return {
            "topic": topic,
            "summary": "Research completed",
            "sources": []
        }
```

- This agent handles the complete research process.
- You can later add real tools (web search, scraping, etc.).
- It returns a structured research result.

## 7 Create Service Layer

services/research_service.py

```python
from agents.research_agent import ResearchAgent

agent = ResearchAgent()

async def run_research(topic: str, depth: str):
    result = await agent.run(
        topic=topic,
        depth=depth
    )
    return result
```

- Service layer acts as a bridge between FastAPI and the agent.
- You can add additional logic here (validation, logging, caching, etc.).
- Keeps the code clean and modular.

## 8 Connect Agent to FastAPI

main.py

```python
from fastapi import FastAPI
from schemas import ResearchRequest
from services.research_service import run_research

app = FastAPI()

@app.post("/research")
async def research(request: ResearchRequest):
    result = await run_research(
        request.topic,
        request.depth
    )
    return result
```

- Takes input using ResearchRequest schema.
- Calls the service layer which runs the research agent.
- Returns the research result as a JSON response.

## 9 Add Research Result Schema

schemas.py

```python
from pydantic import BaseModel

class Source(BaseModel):
    title: str
    url: str

class ResearchResponse(BaseModel):
    topic: str
    summary: str
    sources: list[Source]
```

Use it in main.py

```python
@app.post(
    "/research",
    response_model=ResearchResponse
)
async def research(request: ResearchRequest):
    return await run_research(
        request.topic,
        request.depth
    )
```

- Defines the structure of the research response.
- Ensures consistent and validated output.
- Helps in API documentation (Swagger UI).

## Complete Request Flow



# Creating FastAPI for Research AI Agents

## # Production-Style Research API

## 10 Add Research Job ID

- For a long-running research task, don't make the client wait for the complete research.
- Create a job ID and return immediately.

main.py

```python
from uuid import uuid4
jobs = {}

@app.post("/research")
async def start_research(request: ResearchRequest):
    job_id = str(uuid4())
    jobs[job_id] = {
        "status": "running",
        "topic": request.topic
    }
    return {
        "job_id": job_id,
        "status": "running"
    }
```

**Key Points:**

- Generate a unique job_id using uuid4().
- Store job status in a dictionary (jobs).
- Return job_id immediately.
- Useful for long-running research tasks.

## 11 Research Status API

main.py

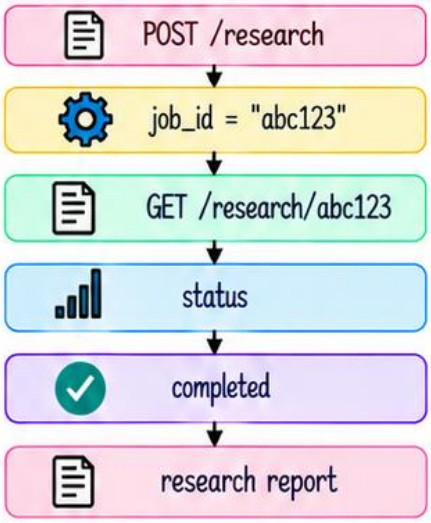
```python
from fastapi import HTTPException

@app.get("/research/{job_id}")
async def get_research_status(job_id: str):
    if job_id not in jobs:
        raise HTTPException(
            status_code=404,
            detail="Research job not found"
        )
    return jobs[job_id]
```

**Key Points:**

- Takes job_id as path parameter.
- Returns current status and result (if completed).
- Raises 404 error if job_id is not found.

### Client Flow (Check Status)




## 12 Add Background Research

- Run the research in the background so the API responds immediately.
- Once research is complete, update the job status with the result.

main.py

```python
from fastapi import BackgroundTasks

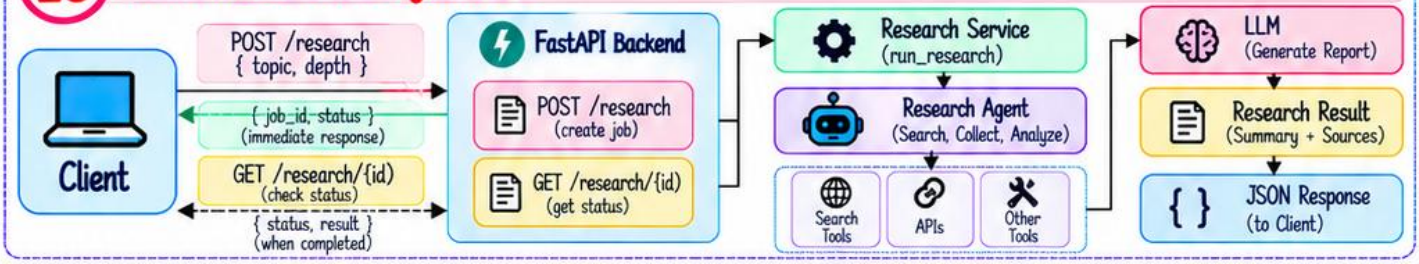
async def execute_research(job_id: str, topic: str):
    result = await run_research(topic, "medium")
    jobs[job_id] = {
        "status": "completed",
        "result": result
    }

@app.post("/research")
async def start_research(
    request: ResearchRequest,
    background_tasks: BackgroundTasks
):
    job_id = str(uuid4())
    jobs[job_id] = {"status": "running"}
    background_tasks.add_task(
        execute_research,
        job_id,
        request.topic
    )
    return {    "job_id": job_id,
                "status": "running"
```

**Key Points:**

- Uses BackgroundTasks to run research in the background.
- Immediately returns job_id to the client.
- Updates job status to "completed" with the result after research finishes.
- Non-blocking and better for long tasks.

## 13 Final Research AI Agent API



TheAgenticAIEngineer

LEARN. BUILD. AUTOMATE. GROW.

# FOLLOW & COMMENT

## TO GET

# THE PDF




@TheAgenticAIEngineer

AI | LLMs | RAG | AGENTS | GUARDRAILS | AI CODING